

Как устроена авторизация, двухфакторка, сброс пароля и сессии

Пошаговый визуальный разбор: что видит пользователь на экране, какой именно запрос в этот момент улетает на сервер (реальная панель Network из Chrome DevTools, крупно — с содержимым запроса, то есть Payload) и что сервер отвечает, и что происходит внутри бэкенда — какие проверки, какой код и какая логика отрабатывают за эти доли секунды. Документ рассчитан на читателя без опыта в разработке.

- Скриншот приложения — то, что видит пользователь в браузере
- Панель Network из Chrome DevTools: Payload — что отправлено, Preview — что получено, Headers — адрес и заголовки
- Тёмные блоки — реальный код бэкенда, с подсветкой

- | | |
|--|---|
| 01 Вход в аккаунт, когда включена 2FA | 06 Редактирование профиля |
| 02 Отключение двухфакторной аутентификации | 07 Забыли пароль: сброс через код из письма |
| 03 Обычный вход (2FA выключена) | 08 Активные сессии и выход с устройств |
| 04 Повторное включение 2FA | 09 Диаграммы последовательности |
| 05 Смена пароля | 10 Что защищает аккаунт: итоги |

Разделы 07–09 добавлены в сентябре 2026: сброс пароля по коду, список активных сессий и сводные диаграммы последовательности.

1 Вход в аккаунт, когда включена 2FA

У аккаунта сейчас включена двухфакторная аутентификация (2FA). Это значит, что одного пароля недостаточно — после него сервер потребует ещё одноразовый код из письма. Смотрим, как это выглядит и что происходит на сервере на каждом шаге.

Как читать скриншоты панели Network

Под каждым действием, которое обращается к серверу, показана настоящая панель Network из Chrome DevTools. Сверху — метод и адрес запроса и код ответа (200 = «всё хорошо»). Ниже по вкладкам: **Payload** — данные, которые браузер отправил серверу (то, что вы «положили в конверт»); **Preview** — данные, которые сервер прислал в ответ; **Headers** — служебные «пометки на конверте» (адрес, метод, заголовок Authorization с токеном). Если запрос ничего не отправляет в теле (например, просто «дай мой профиль»), вкладки Payload у него нет — тогда показан Headers.

Шаг 1 — заполняем форму входа FRONTEND

Sign In

Access the platform admin console

Work Email

Password [Forgot Password?](#)

Sign in

Форма входа заполнена, кнопка «Sign in» ещё не нажата. В Network при этом пусто — ничего не отправлено.

Пока вы печатаете email и пароль, в сеть ничего не уходит — проверяется только на глаз, что поля не пустые. Реальная проверка происходит только после нажатия «Sign in».

← Back

Enter 2FA otp

We sent a 6 digit otp to shipinomaksim@gmail.com.

Verify 2FA OTP

Браузер сам переключился на страницу ввода кода: сервер ответил, что для входа нужен второй шаг.

POST

http://localhost:4000/auth/login

200 OK

→ Payload — что браузер отправил на сервер (тело запроса)

Elements

Console

Sources

Network

Performance

Memory

Application

Security

>>

Name

login

×

Headers

Payload

Preview

Response

Initiator

Timing

▼ Request payload

View source

{email: "shipinomaksim@gmail.com", password: "33333333"}
email: "shipinomaksim@gmail.com"
password: "33333333"

← Preview — что сервер ответил

⌕

Elements

Console

Sources

Network

Performance

Memory

Application

Security

>>

Name

✕

Headers

Payload

Preview

Response

Initiator

Timing

login

▼ {twoFactorRequired: true,...}

message: "Otp send successfully"

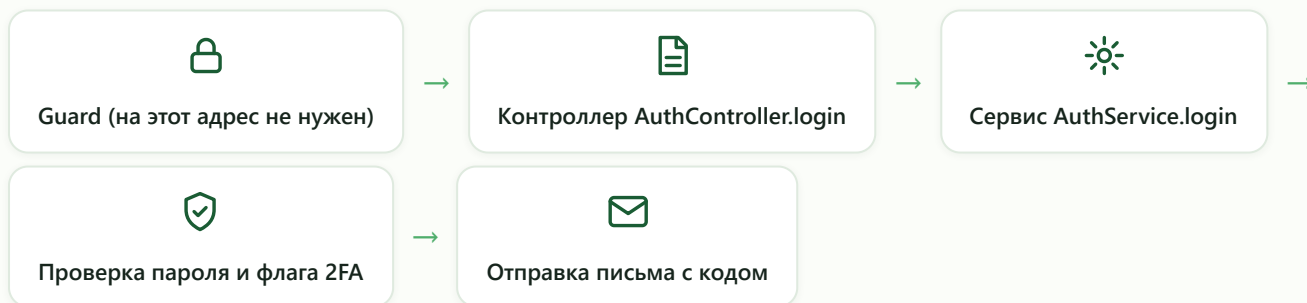
twoFactorRequired: true

twoFactorToken: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VysWw"

Что улетело на сервер и что вернулось

Payload — всего два поля: `email` и `password`. Это тело запроса: маленький JSON-документ, который браузер положил в запрос. Пароль виден открытым текстом — внутри запроса так и должно быть: в пути его защищает HTTPS-соединение (в боевой среде оно обязательно; здесь, на `localhost`, шифрования нет и это нормально для разработки), а на сервере он сразу превращается в хэш для сравнения и нигде не сохраняется. **Preview** — в ответе **нет** ключей доступа: только `twoFactorRequired: true`, `message` и `twoFactorToken` — длинная подписанная строка, «билет на второй шаг».

Это ключевой момент: запрос `login` вернул **200 OK** (пароль верный!), но без токенов доступа. Вместо них сервер прислал короткоживущий «билет на второй шаг». Разберём, что произошло внутри:



Простыми словами. Вход не защищён "охранником" (Guard'ом), потому что на этом шаге пользователь ещё не залогинен — охранять нечего, наоборот, сюда должен суметь достучаться кто угодно с логином и паролем. А вот дальше контроллер (это как диспетчер: принял запрос, передал по адресу и вернул ответ) отдаёт всю работу сервису. Сервис:

- 1 Находит пользователя по `email` в базе данных.
- 2 Сверяет пароль с тем, что хранится в базе (в базе лежит не сам пароль, а его необратимый «отпечаток» — хэш; сравнение идёт по отпечаткам).
- 3 Видит, что у аккаунта включена 2FA — и вместо выдачи токенов генерирует 6-значный код, сохраняет его (тоже в виде хэша, не в открытом виде!) и отправляет на почту через сервис Resend.

- 4 Подписывает специальный короткоживущий токен (живёт 15 минут) — это тот самый «билет на второй шаг», который видно в ответе на скриншоте как `twoFactorToken`.

```
// backend/src/auth/auth.service.ts
async login(email: string, password: string, context = {}) {
  const user = await this.usersService.findByEmail(email);

  // Сравнение с "фиктивным" хэшем, даже если пользователь не найден -
  // чтобы ответ занимал одинаковое время в обоих случаях (защита от
  // подбора существующих email по разнице в скорости ответа).
  const passwordMatches = await bcrypt.compare(
    password,
    user?.passwordHash ?? ABSENT_USER_PASSWORD_HASH,
  );

  if (!user || !passwordMatches) {
    throw new UnauthorizedException('Invalid email or password');
  }

  if (user.twoFactorEnabled) {
    // Не выдаём токены сразу - создаём одноразовый код и билет
    // на второй шаг проверки.
    const result = await this.twoFactorCreateOtpToken(user.id);
    return {
      twoFactorRequired: true,
      twoFactorToken: result.twoFactorToken,
      message: result.message,
    };
  }

  const tokens = await this.issueTokenPair(user, context);
  return { ...tokens, userType: user.userType };
}
```

Почему пароль сравнивают даже для несуществующего email?

Это защита от «угадывания» зарегистрированных адресов. Если бы для несуществующего email сервер отвечал мгновенно, а для существующего - с небольшой задержкой (на сравнение пароля), по одной только скорости ответа можно было бы понять, зарегистрирован ли email в системе. Поэтому сравнение хэшей выполняется всегда, даже вхолостую.

Шаг 3 — вводим код из письма FRONTEND

[← Back](#)

Enter 2FA otp

We sent a 6 digit otp to shipinomaksim@gmail.com.

5

5

0

3

1

3

Verify 2FA OTP

Шестизначный код, только что пришедший на почту, введён в поле.

Шаг 4 — код проверен, доступ открыт FRONTEND BACKEND

Platform

Dashboard

Sellers

Commision & Payouts

Global Catalog

Delivery Network

Customers

Orders

Supports

Marketing

Reports

Users & Permissions

Settings

Shipino Maksim
shipinomaksim@gmail.com

Q search store, order, agent...

DashboardPage

После кнопки «Verify»: вход выполнен, открылся дашборд.

200 OK

The screenshot shows the Chrome DevTools Network tab. The 'Payload' sub-tab is selected, displaying the request payload for the '2fa-verify-login-otp' request. The payload is a JSON object containing 'otp' and 'twoFactorToken' fields. A 'View source' button is present next to the 'Request payload' header.

Name	Headers	Payload	Preview	Response	Initiator	Timing
2fa-verify-login-otp		Request payload View source {, ...} otp: "550313" twoFactorToken: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQ"				
me						
me						

[illegible]

Payload — два поля: `otp` (шесть цифр из письма) и `twoFactorToken` (тот самый «билет» из предыдущего ответа). Сервер ничего не помнит между запросами, поэтому браузер сам возвращает билет вместе с кодом. **Preview** — теперь в ответе настоящие ключи: `accessToken`, `refreshToken` и `userType`.

GET

http://localhost:4000/auth/me

200 OK

→ Headers — адрес, метод и заголовки запроса (в том числе Authorization)

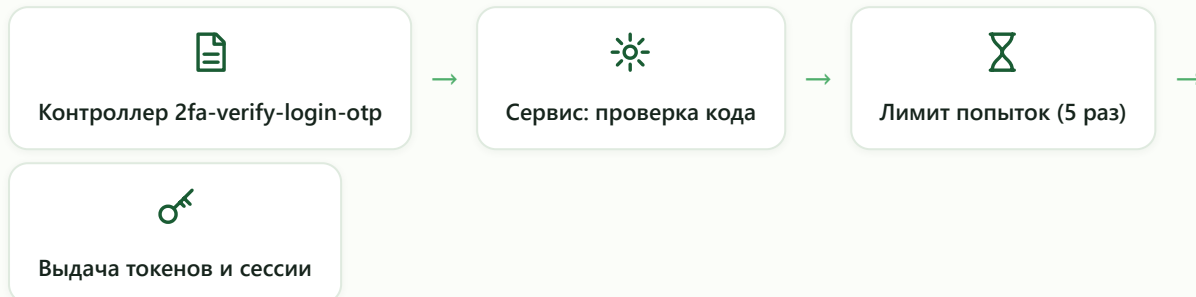
Name	Headers	Preview	Response	Initiator	Timing
2fa-verify-login-otp	▼ General				
me	Request URL		http://localhost:4000/auth/me		
me	Request Method		GET		
	Status Code		200 OK		
	Remote Address		:::1:4000		
	Referrer Policy		strict-origin-when-cross-origin		
	► Response headers (24)				
	▼ Request headers	☐ Raw			
	Accept		application/json, text/plain, */*		
	Accept-Encoding		gzip, deflate, br, zstd		
	Accept-Language		uk-UA,uk;q=0.9,en-US;q=0.8,en;q=0.7		
	Authorization		Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJjbXU1cXZtNGUwMDAwamN2Nm04NDI6ZTJpIiwiaWF0IjZaGlwaw5vbWFRc2ltQGdtYWlsLmNvbSIsInVzZXJueXB1IjoieUEExBVEZPuk1fQURN SU4iLCJzaWQiOiJnNGFoMXhzbGw5YmFoMXIxZHI1NTY2bzUiLCJpYXQ iOjE3ODk4MzExNDIsImV4cCI6MTc4OTgzMjA0Mn0.18FjwANQcEqzUI NMGINGhEJwzM9v7SrNiGPefRxosa0		
	Connection		keep-alive		

← Preview — что сервер ответил

Name	Headers	Preview	Response	Initiator	Timing
2fa-verify-login-otp		▼	{id: "cmu5qvm4e0000jcv6j849ze2i", email: "shipinomaksim@gmail.com"		
me			createdAt: "2026-09-17T16:29:16.096Z"		
me			email: "shipinomaksim@gmail.com"		
			fullName: "Shipino Maksim"		
			id: "cmu5qvm4e0000jcv6j849ze2i"		
			phone: "3850006879"		
			profileImage: "https://github.com/shadcn.png"		
			status: "ACTIVE"		
			twoFactorEnabled: true		
			updatedAt: "2026-09-18T13:36:46.770Z"		
			userType: "PLATFORM_ADMIN"		

Как сервер узнаёт, кто спрашивает

У этого запроса нет тела — браузер просто просит «дай мой профиль», поэтому вкладка **Payload** отсутствует. Зато в **Headers** видно главное: заголовок `Authorization: Bearer <accessToken>` — тот самый ключ доступа, выданный секундой раньше. В **Preview** приходят данные профиля (имя, email, телефон, тип пользователя и т.д.) — без пароля и секретов 2FA.



Простыми словами. Сервер достаёт из базы последний код, который предназначен именно «для входа» (а не, например, для включения 2FA — у каждого кода есть метка «назначения», чтобы код от одного действия нельзя было подсунуть в другое). Сверяет введённое с хэшем. Если код неверный — увеличивает счётчик неудачных попыток; после пятой неудачной попытки код перестаёт приниматься вообще, даже если ввести его правильно — это защита от перебора всех 900 000 возможных комбинаций.

```
// backend/src/auth/auth.service.ts

async twoFactorVerifyLoginOtp(twoFactorToken: string, otp: string, context = {}) {
  // 1. Билет на второй шаг должен быть подлинным и ещё не истёкшим.
  let decode: TwoFactorTokenPayload;
  try {
    decode = await this.jwtService.verifyAsync(twoFactorToken, {
      secret: this.config.getOrThrow<string>('JWT_2FA_SECRET'),
    });
  } catch {
    throw new BadRequestException('Session is Expired');
  }

  const user = await this.prisma.user.findUnique({ where: { id: decode.userId } });
  if (user.status !== UserStatus.ACTIVE) {
    throw new ForbiddenException('This account is not active');
  }

  // 2. Проверка самого кода: назначение "LOGIN", лимит попыток, хэш.
  await this.verifyOtp(user.id, TwoFactorOtpPurpose.LOGIN, otp);

  // 3. Всё сошлось - только теперь выдаём настоящие токены доступа.
  const token = await this.issueTokenPair(user, context);
  return { ...token, userType: user.userType };
}
```

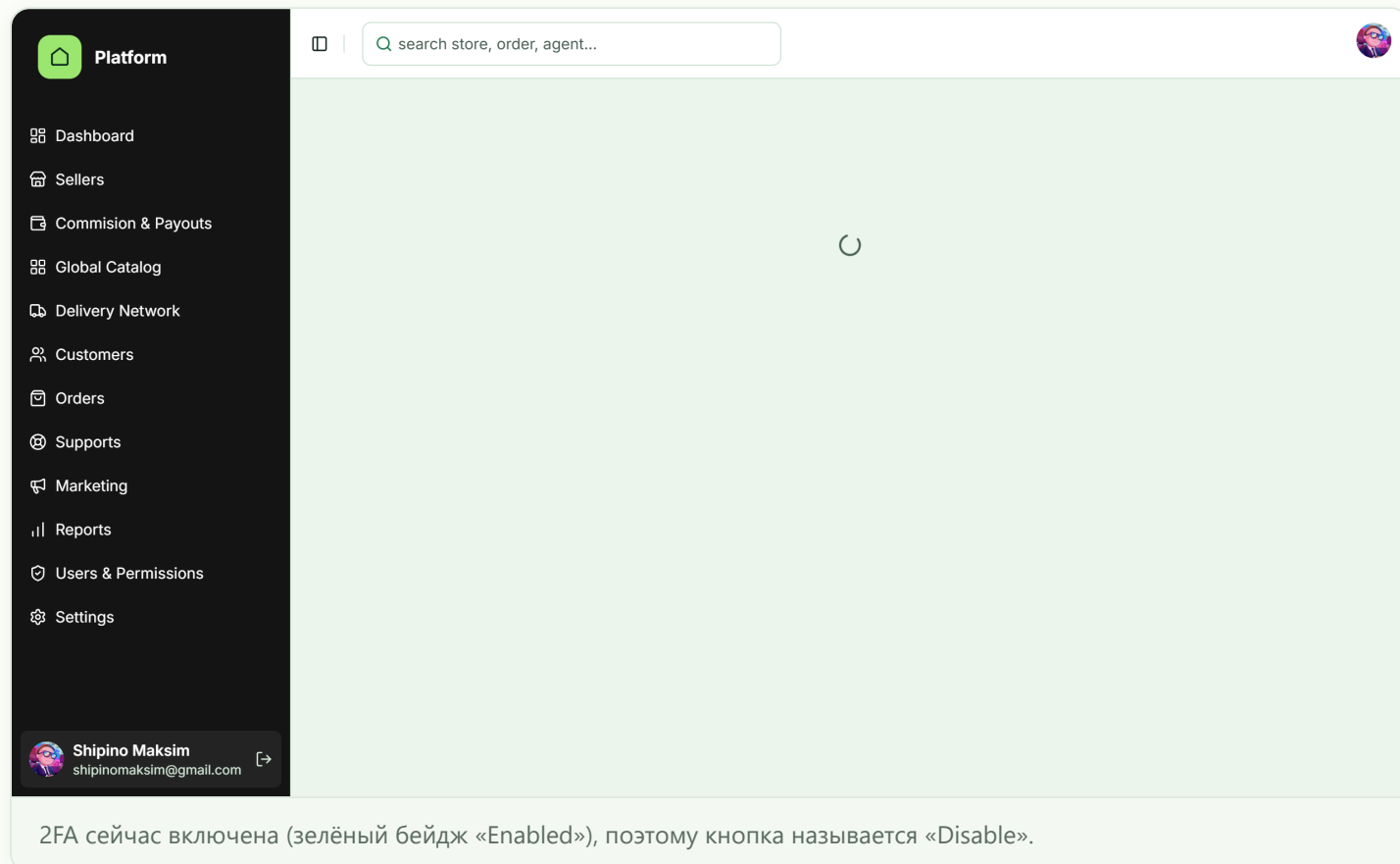
А что за два запроса "me" подряд?

Первый - GET /auth/me сразу после входа: страница дашборда запрашивает профиль, чтобы показать имя и аватар. Второй запрос с кодом 304 - это React в строгом режиме разработки (Strict Mode) намеренно повторяет эффект, чтобы разработчики ловили побочные эффекты; сервер отвечает "содержимое не изменилось", лишней нагрузки это не создаёт.

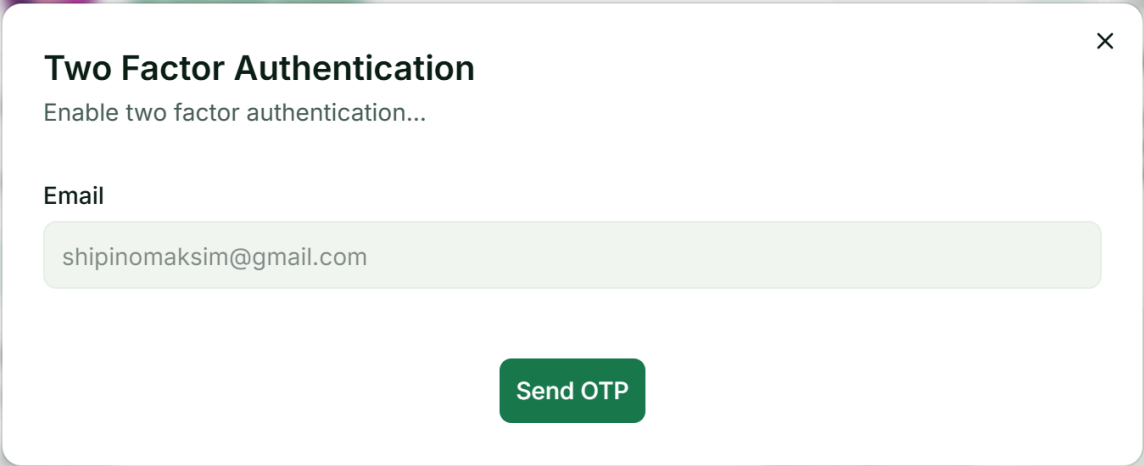
2 Отключение двухфакторной аутентификации

Раздел «Мой аккаунт» → «Password & Security». Включение и выключение 2FA технически — один и тот же переключатель на сервере: он всегда ставит значение, противоположное текущему. Поэтому что «включить», что «выключить» проходит через один и тот же код, только с разным начальным состоянием.

Шаг 1 — страница аккаунта **FRONTEND**



Шаг 2 — открываем диалог **FRONTEND**



Two Factor Authentication ×

Enable two factor authentication...

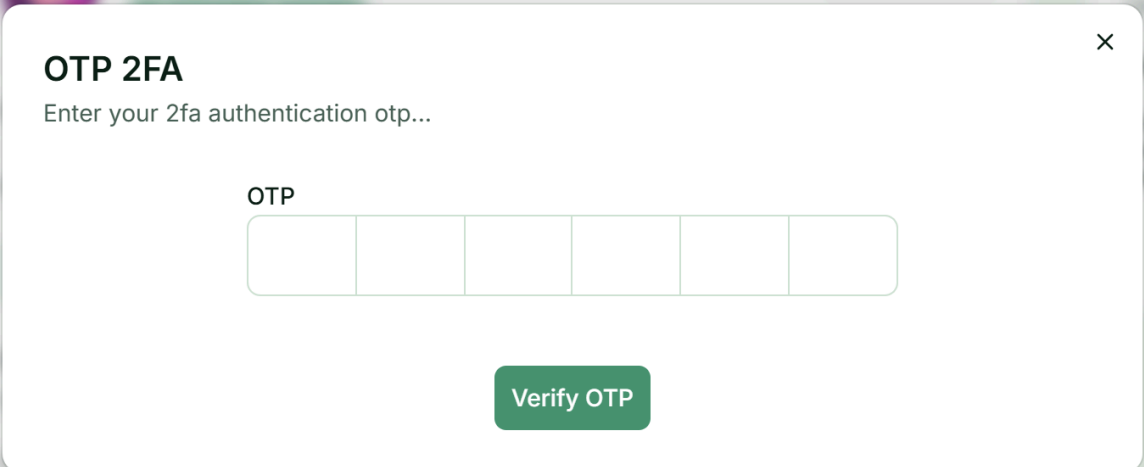
Email

shipinomaksim@gmail.com

Send OTP

Диалог из двух шагов: сначала подтверждаем e-mail и просим код.

Шаг 3 — код отправлен **FRONTEND** **BACKEND**



OTP 2FA ×

Enter your 2fa authentication otp...

OTP

Verify OTP

Код создан и отправлен на почту; форма переключилась на ввод шести цифр.

POST

http://localhost:4000/auth/2fa-generate-otp

200 OK

→ Headers — адрес, метод и заголовки запроса (в том числе Authorization)

Elements

Console

Sources

Network

Performance

Memory

Application

Security

>>

5

⚙

⋮

Name

✕

Headers

Preview

Response

Initiator

Timing

2fa-generate-otp

▼ General

Request URL

http://localhost:4000/auth/2fa-generate-otp

Request Method

POST

Status Code

200 OK

Remote Address

[::1]:4000

Referrer Policy

strict-origin-when-cross-origin

▼ Response headers

☐ Raw

Access-Control-Allow-Credentials

true

Access-Control-Allow-Origin

http://localhost:3002

Connection

keep-alive

Content-Length

50

Content-Security-Policy

default-src 'self';base-uri 'self';font-src 'self' https: data:;form-action 'self';frame-ancestors 'self';img-src 'self' data:;object-src 'none';script-src 'self';script-src-attr 'none';style-src 'self' https: 'unsafe-inline';upgrade-insecure-requests

Content-Type

application/json; charset=utf-8

← Preview — что сервер ответил

Elements

Console

Sources

Network

Performance

Memory

Application

Security

>>

Name

✕

Headers

Preview

Response

Initiator

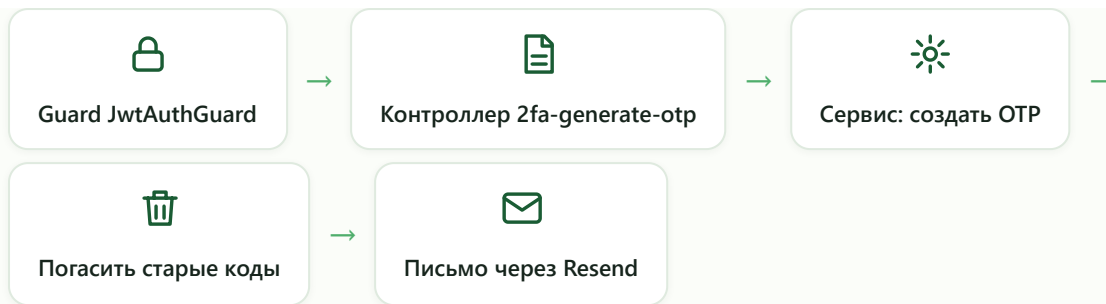
Timing

2fa-generate-otp

▼ {success: true, message: "Otp send successfully"}
message: "Otp send successfully"
success: true

Что улетело на сервер и что вернулось

Тела у запроса нет (нечего передавать — сервер и так знает, кто вы по заголовку Authorization), хотя метод POST: он здесь просто означает «выполни действие», поэтому показан **Headers**: адрес, метод POST и заголовок Authorization: Bearer В **Preview** — только success: true и сообщение Otp send successfully . Обратите внимание: самого кода в ответе **нет** — он уходит исключительно на почту, иначе любой, кто перехватит ответ, получил бы и код.



Простыми словами про Guard. В отличие от входа, этот запрос требует, чтобы человек уже был залогинен — поэтому перед контроллером стоит «охранник» `JwtAuthGuard`. Он смотрит на заголовок `Authorization: Bearer ...`, присланный браузером, проверяет подпись токена и, если всё в порядке, сверяется с базой — жива ли ещё сессия (её могли принудительно отозвать, например, при выходе с другого устройства). Если что-то не так — запрос отклоняется ошибкой 401 ещё до того, как выполнится хоть строчка бизнес-логики.

```
// backend/src/common/guards/jwt-auth.guard.ts
@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {}

// backend/src/auth/strategies/jwt.strategy.ts
async validate(payload: JwtAccessPayload): Promise<AuthenticatedUser> {
  const session = await this.prisma.userSession.findUnique({
    where: { id: payload.sid },
    include: { user: true },
  });

  // Сессию могли отозвать вручную (например, кнопкой "выйти со всех устройств") - тогда токен всё ещё "живой" по подписи, но сервер всё равно откажет.
  if (!session || session.revokedAt || session.expiresAt <= new Date()) {
    throw new UnauthorizedException('Session is no longer valid');
  }
  if (session.user.status !== UserStatus.ACTIVE) {
    throw new UnauthorizedException('This account is not active');
  }

  return { userId: session.userId, email: session.user.email,
    userType: session.user.userType, sessionId: session.id };
}
```

Шаг 4 — вводим код

FRONTEND

OTP 2FA

Enter your 2fa authentication otp...

OTP

9

3

9

9

7

0

Verify OTP

Код из письма введён, готовы нажать «Verify OTP».

Шаг 5 — 2FA выключена

FRONTEND

BACKEND

Platform

Dashboard

Sellers

Commision & Payouts

Global Catalog

Delivery Network

Customers

Orders

Supports

Marketing

Reports

Users & Permissions

Settings

Shipino Maksim

shipinomaksim@gmail.com

search store, order, agent...

My account

Your profile, password, two-factor authentication, and where you're signed in

Shipino Maksim

PLATFORM_ADMIN

Edit profile

Email

shipinomaksim@gmail.com

Phone

3850006879

Role & Access

Update your profile details

Role

-

Full access to every module -stores, catalog, payouts, delivery, network, and platform settings.

View permission matrix →

Password & Security

Manage your password and two-factor authentication

Password

Last changed 2 months ago

Change Password

Two-factor authentication

Extra code required at login, in addition to your password

Disabled

Enable

Active sessions

Devices where you're currently signed in

3 devices signed in

Two-factor disabled

На карточке появляется актуальное состояние переключателя — 2FA выключена.

POST

http://localhost:4000/auth/2fa-verify-enable

200 OK

→ Payload — что браузер отправил на сервер (тело запроса)

ElementsConsoleSourcesNetworkPerformanceMemoryApplicationSecurity>>

NameXHeadersPayloadPreviewResponseInitiatorTiming

2fa-verify-enable

Request payloadView source

{otp: "939970"}
otp: "939970"

← Preview — что сервер ответил

ElementsConsoleSourcesNetworkPerformanceMemoryApplicationSecurity>>

NameXHeadersPayloadPreviewResponseInitiatorTiming

2fa-verify-enable

{success: true, message: "Two-factor disabled", data: {twoFactorEnabled: false}}

data: {twoFactorEnabled: false}
message: "Two-factor disabled"
success: true

Что улетело на сервер и что вернулось

Payload — одно поле: `otp`. В запросе нет ни слова «выключить»: сервер сам решает, что делать, и просто переключает значение на противоположное. **Preview**: `message: "Two-factor disabled"` и `data.twoFactorEnabled: false` — итоговое состояние, которое сервер записал в базу.

```
// backend/src/auth/auth.service.ts

async twoFactorVerifyEnable(userId: string, otp: string) {
  const user = await this.prisma.user.findUnique({ where: { id: userId } });

  // Код обязательно должен быть с меткой ENABLE_TOGGLE - код, присланный
  // для входа, здесь не примут, даже если он ещё действителен.
  await this.verifyOtp(userId, TwoFactorOtpPurpose.ENABLE_TOGGLE, otp);

  const updatedUser = await this.prisma.user.update({
    where: { id: user.id },
    data: { twoFactorEnabled: !user.twoFactorEnabled }, // переключатель
  });

  return {
    success: true,
    message: `Two-factor ${updatedUser.twoFactorEnabled ? 'enabled' : 'disabled'}`,
    data: { twoFactorEnabled: updatedUser.twoFactorEnabled },
  };
}
```

Зачем коду "метка назначения"?

Коды для входа и коды для включения/выключения 2FA генерируются одной и той же функцией и хранятся в одной таблице. Без метки назначения (purpose) код, присланный для входа, можно было бы подсунуть в форму отключения 2FA - и наоборот. Метка гарантирует, что код, полученный для одной цели, не сработает для другой.

3

Сейчас 2FA временно выключена (предыдущий раздел). Для сравнения — вот как выглядит вход без второго шага: пароль подошёл — и сразу выданы токены доступа.

Шаг 1 — выходим и заходим заново FRONTEND

Sign In

Access the platform admin console

Work Email

Password [Forgot Password?](#)

Sign in

Снова вводим email и пароль — те же, что и раньше.

Platform

Dashboard

Sellers

Commision & Payouts

Global Catalog

Delivery Network

Customers

Orders

Supports

Marketing

Reports

Users & Permissions

Settings

Shipino Maksim
shipinomaksim@gmail.com

search store, order, agent...

DashboardPage

Пароль подошёл — и сразу дашборд, без промежуточного шага с кодом.

POST

http://localhost:4000/auth/login

200 OK

→ Payload — что браузер отправил на сервер (тело запроса)

Elements

Console

Sources

Network

Performance

Memory

Application

Security

>>

Name

login

me

me

Headers

Payload

Preview

Response

Initiator

Timing

Request payload

View source

{email: "shipinomaksim@gmail.com", password: "33333333"}
email: "shipinomaksim@gmail.com"
password: "33333333"

← Preview — что сервер ответил

A screenshot of the Chrome DevTools Network tab. The 'Network' tab is selected at the top. On the left, a list of network requests is shown, with 'login' selected. The 'Preview' sub-tab is active, displaying the response body of the selected request. The response is a JSON object containing 'accessToken', 'refreshToken', and 'userType'.

Сравните с входом при включённой 2FA

Payload тот же самый: `email` и `password`. Разница — в **Preview**: вместо «билета на второй шаг» сервер сразу присылает `accessToken`, `refreshToken` и `userType`.

Это тот же самый метод `AuthService.login`, что и в разделе 1, просто условие `if (user.twoFactorEnabled)` в этот раз не срабатывает — и код сразу переходит к выдаче токенов:

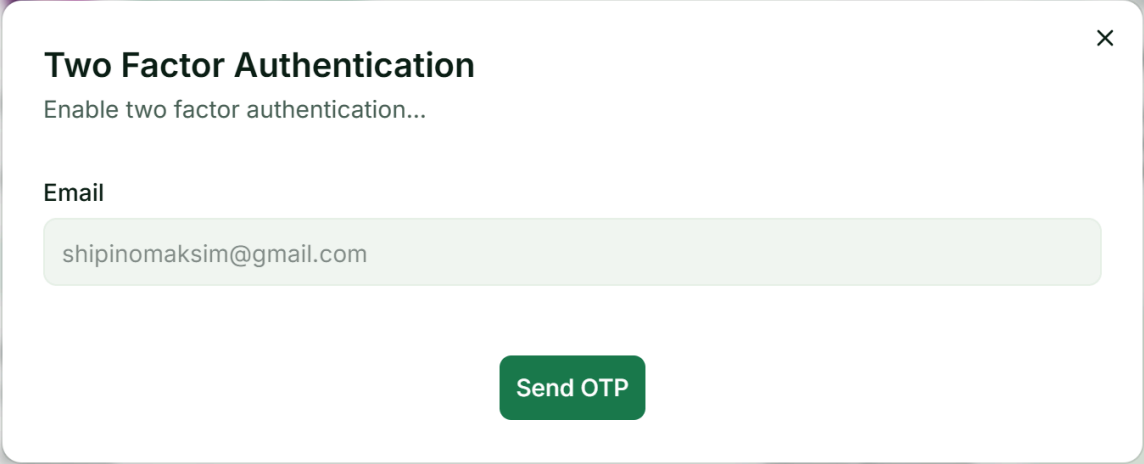
```
const tokens = await this.issueTokenPair(user, context);
return { ...tokens, userType: user.userType };
```

Что именно выдаётся

Два токена: короткоживущий access-токен (по нему сервер узнаёт "кто вы" при каждом запросе, живёт 15 минут) и долгоживущий refresh-токен (нужен только для того, чтобы незаметно получить новый access-токен, когда старый истечёт). Плюс в базе создаётся запись о сессии - устройство, IP, время входа - её видно в разделе «Active sessions».

4 Повторное включение 2FA

Возвращаем защиту обратно — ровно тот же переключатель, что и в разделе 2, только в другую сторону.



Two Factor Authentication ×

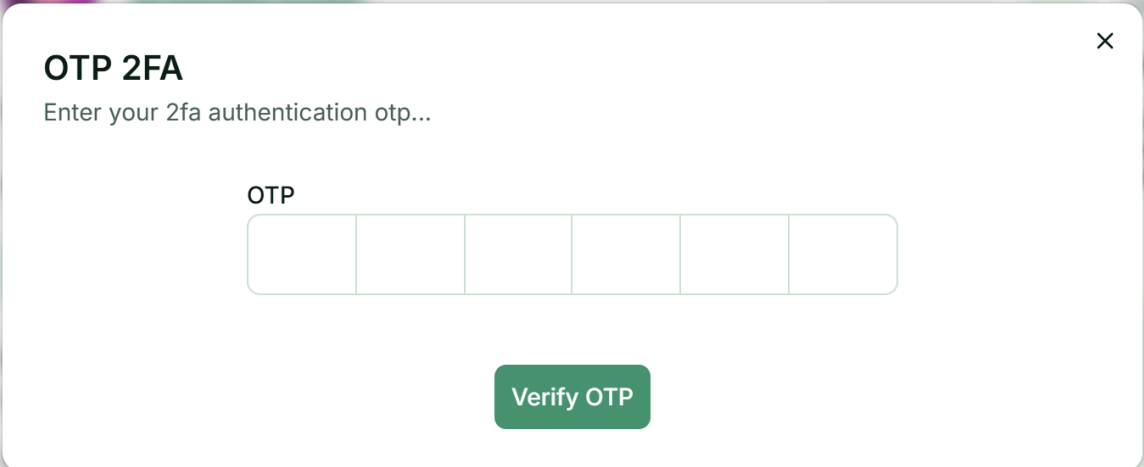
Enable two factor authentication...

Email

shipinomaksim@gmail.com

Send OTP

Кнопка снова называется «Enable» — 2FA сейчас выключена.



OTP 2FA ×

Enter your 2fa authentication otp...

OTP

Verify OTP

Новый код сгенерирован и отправлен на почту.

POST

http://localhost:4000/auth/2fa-generate-otp

200 OK

← Preview — что сервер ответил

⌵

Elements

Console

Sources

Network

Performance

Memory

Application

Security

>>

Name

2fa-generate-otp

✕

Headers

Preview

Response

Initiator

Timing

▼ {success: true, message: "Otp send successfully"}
message: "Otp send successfully"
success: true

OTP 2FA

Enter your 2fa authentication otp...

OTP

7

2

1

1

4

5

Verify OTP

Код введён.

Platform

Dashboard

Sellers

Commision & Payouts

Global Catalog

Delivery Network

Customers

Orders

Supports

Marketing

Reports

Users & Permissions

Settings

Shipino Maksim

shipinomaksim@gmail.com

Q search store, order, agent...

My account

Your profile, password, two-factor authentication, and where you're signed in

Shipino Maksim

PLATFORM_ADMIN

Edit profile

Email

shipinomaksim@gmail.com

Phone

3850006879

Role & Access

Update your profile details

Role

-

Full access to every module -stores, catalog, payouts, delivery, network, and platform settings.

View permission matrix →

Password & Security

Manage your password and two-factor authentication

Password

Last changed 2 months ago

Change Password

Two-factor authentication

Extra code required at login, in addition to your password

Enabled

Disable

Active sessions

Devices where you're currently signed in

3 devices signed in

Two-factor enabled

2FA снова включена.

POSThttp://localhost:4000/auth/2fa-verify-enable200 OK

→ Payload — что браузер отправил на сервер (тело запроса)

ElementsConsoleSourcesNetworkPerformanceMemoryApplicationSecurity>>

Name

2fa-verify-enable

✕

Headers

Payload

Preview

Response

Initiator

Timing

▼ Request payload

View source

▼ {otp: "721145"}

otp: "721145"

← Preview — что сервер ответил

⌵		Elements	Console	Sources	Network	Performance	Memory	Application	Security	>>
Name	✕	Headers	Payload	Preview	Response	Initiator	Timing			
2fa-verify-enable	▼	{success: true, message: "Two-factor enabled", data: {twoFactorEnabled: true}}								
	▶	data: {twoFactorEnabled: true}								
		message: "Two-factor enabled"								
		success: true								

Тот же запрос — противоположный результат

В **Payload** снова только `otp` , ровно как при выключении. Отличается лишь **Preview**: теперь сервер вернул `twoFactorEnabled: true` .

Итог

Код внутри - буквально тот же самый метод `twoFactorVerifyEnable` из раздела 2. Он не спрашивает "включить" или "выключить" - он просто ставит противоположное значение тому, что было. Именно поэтому фронтенд сам решает, какую надпись показать на кнопке ("Enable" или "Disable"), глядя на текущее состояние.

5 Смена пароля

Отдельная форма, отдельный адрес на сервере, ничего общего с редактированием профиля — и это осознанное решение, разберём почему.

Шаг 1 — заполняем форму **FRONTEND**

×

Change Password

Update your password here...

Current Password

.....

👁

Password

.....

👁

Confirm Password

.....

👁

Save changes

Текущий пароль, новый пароль и его подтверждение.

Platform

Dashboard

Sellers

Commision & Payouts

Global Catalog

Delivery Network

Customers

Orders

Supports

Marketing

Reports

Users & Permissions

Settings

Shipino Maksim

shipinomaksim@gmail.com

search store, order, agent...

My account

Your profile, password, two-factor authentication, and where you're signed in

Shipino Maksim

PLATFORM_ADMIN

Edit profile

Email

shipinomaksim@gmail.com

Phone

3850006879

Role & Access

Update your profile details

Role

-

Full access to every module -stores, catalog, payouts, delivery, network, and platform settings.

View permission matrix →

Password & Security

Manage your password and two-factor authentication

Password

Last changed 2 months ago

Change Password

Two-factor authentication

Extra code required at login, in addition to your password

Enabled

Disable

Active sessions

3 devices signed in

Devices where you're currently signed in

Password changed successfully

Уведомление «Password changed successfully».

PUThttp://localhost:4000/users/change-password200 OK

→ Payload — что браузер отправил на сервер (тело запроса)

ElementsConsoleSourcesNetworkPerformanceMemoryApplicationSecurity

Name

change-password

HeadersPayloadPreviewResponseInitiatorTiming

Request payload

View source

{currentPassword: "33333333", password: "TempDocgen2026!"}

currentPassword: "33333333"

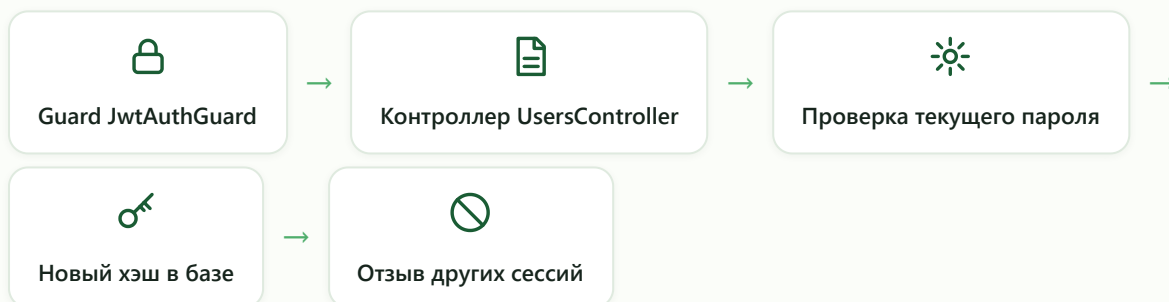
password: "TempDocgen2026!"

← Preview — что сервер ответил

	Elements	Console	Sources	Network	Performance	Memory	Application	Security	>>
Name	×	Headers	Payload	Preview	Response	Initiator	Timing		
 change-password				▼ {user: {id: "cmu5qvm4e0000jcv6j849ze2i", email: "shipinomaksim@gmail.com"}, message: "Password changed successfully", success: true} ▶ user: {id: "cmu5qvm4e0000jcv6j849ze2i", email: "shipinomaksim@gmail.com"}					

Что улетело на сервер и что вернулось

Payload — два поля: `currentPassword` (текущий пароль) и `password` (новый). Поле «подтверждение пароля» из формы на сервер **не отправляется** — совпадение двух полей проверяет сам браузер ещё до запроса. **Preview:** `success: true`, сообщение и обновлённый профиль `user` (без пароля и без хэша).



```
// backend/src/users/users.service.ts

async changePassword(userId: string, dto: ChangePasswordDto, currentSessionId?: string) {
  const existingUser = await this.prisma.user.findUnique({ where: { id: userId } });

  const currentPasswordMatches = await bcrypt.compare(
    dto.currentPassword,
    existingUser.passwordHash,
  );

  if (!currentPasswordMatches) {
    throw new UnauthorizedException('Current password is incorrect');
  }

  const passwordHash = await bcrypt.hash(dto.password, 10);
  const user = await this.prisma.user.update({
    where: { id: userId },
    data: { passwordHash },
  });

  // Если пароль скомпрометирован и им уже кто-то пользуется на другом
  // устройстве - эта смена пароля выкидывает его оттуда. Своё же
  // текущее устройство (с которого только что сменили пароль) не трогаем.
  await this.prisma.userSession.updateMany({
    where: { userId, revokedAt: null, id: { not: currentSessionId } },
    data: { revokedAt: new Date() },
  });

  return { user, message: 'Password changed successfully', success: true };
}
```

Зачем спрашивать текущий пароль, если человек и так уже вошёл в аккаунт?

Потому что "вошёл в аккаунт" и "знает пароль" - разные вещи. Например, если кто-то украл активную сессию (токен) через уязвимость в браузере или получил доступ к разблокированному телефону, он сможет открыть настройки, но не будет знать текущий пароль. Требование ввести его ещё раз не даёт злоумышленнику тихо перехватить аккаунт себе, сменив пароль без ведома владельца.

Почему это отдельная форма, а не часть "Редактировать профиль"?

У формы редактирования профиля (имя, e-mail, телефон) на сервере нет ни единого поля для пароля - оно физически отсутствует в структуре допустимых данных этого запроса. Значит, даже теоретически через эту форму пароль сменить нельзя. Единственный путь изменить пароль - через отдельный адрес, который дополнительно требует текущий пароль. Это уменьшает количество мест, где можно случайно (или намеренно) утечь эту возможность.

Редактирование профиля

Имя, e-mail, телефон, фото — самая обычная форма «обновить данные».

✕

Edit profile

Update your profile details

Full name

shipinomaksim@gmail.com

Email

3850006879

Phone

https://github.com/shadcn.png

Profile image

Save changes

Имя изменено, кнопка «Save changes» стала активной (до первого изменения она заблокирована).

Platform

Dashboard

Sellers

Commision & Payouts

Global Catalog

Delivery Network

Customers

Orders

Supports

Marketing

Reports

Users & Permissions

Settings

Maksim Shipino (docs demo)
shipinomaksim@gmail.com

search store, order, agent...

My account

Your profile, password, two-factor authentication, and where you're signed in

Maksim Shipino (docs demo)

PLATFORM_ADMIN

Edit profile

Email

shipinomaksim@gmail.com

Phone

3850006879

Role & Access

Update your profile details

Role

-

Full access to every module -stores, catalog, payouts, delivery, network, and platform settings.

View permission matrix →

Password & Security

Manage your password and two-factor authentication

Password

Last changed 2 months ago

Change Password

Two-factor authentication

Extra code required at login, in addition to your password

Enabled

Disable

Active sessions

Devices where you're currently signed in

3 devices signed in

Updated successfully

Уведомление «Updated successfully».

PUT

http://localhost:4000/users/me

200 OK

→ Payload — что браузер отправил на сервер (тело запроса)

ElementsConsoleSourcesNetworkPerformanceMemoryApplicationSecurity>>

NameXHeadersPayloadPreviewResponseInitiatorTiming

me

▼ Request payloadView source

▼ {fullName: "Maksim Shipino (docs demo)", email: "shipinomaksim@gmail.com", email: "shipinomaksim@gmail.com", fullName: "Maksim Shipino (docs demo)", phone: "3850006879", profileImage: "https://github.com/shadcn.png"}

← Preview — что сервер ответил

ElementsConsoleSourcesNetworkPerformanceMemoryApplicationSecurity>>

NameXHeadersPayloadPreviewResponseInitiatorTiming

me

▼ {user: {id: "cmu5qvm4e0000jcv6j849ze2i", email: "shipinomaksim@gmail.com", message: "Updated successfully", success: true}, user: {id: "cmu5qvm4e0000jcv6j849ze2i", email: "shipinomaksim@gmail.com"}}

Что улетело на сервер и что вернулось

Payload — ровно четыре поля: `fullName`, `email`, `phone` и `profileImage`. Поля «пароль» здесь нет и быть не может (см. предыдущий раздел). **Preview**: `success: true`, `message: "Updated successfully"` и свежий профиль `user`.

```
// backend/src/users/users.service.ts
async update(userId: string, dto: UpdateUserDto): Promise<UpdateUserResponse> {
  try {
    const user = await this.prisma.user.update({
      where: { id: userId },
      data: { ...dto }, // только fullName, email, phone, profileImage
      omit: { passwordHash: true, twoFactorSecret: true },
    });
    return { user, message: 'Updated successfully', success: true };
  } catch (error) {
    // Кто-то уже использует этот e-mail - понятная ошибка вместо "500".
    if (error instanceof Prisma.PrismaClientKnownRequestError && error.code === 'P2002') {
      throw new ConflictException('Email is already in use');
    }
    throw error;
  }
}
```

Почему кнопка «Save changes» была неактивна до правки?

Фронтенд сравнивает текущие значения полей с исходными и блокирует сохранение, пока нет ни одного реального изменения - это просто избавляет от лишнего запроса на сервер, когда сохранять на самом деле нечего.

А что если ввести поле, которого нет в форме?

Не получится даже искусственно - на входе в сервер стоит общий фильтр (ValidationPipe с whitelist), который вообще отбрасывает любые поля, не описанные в разрешённом списке для этого запроса, и с настройкой forbidNonWhitelisted вернёт ошибку 400, если такое поле обнаружено.

7 Забыли пароль: сброс через код из письма

Пользователь не помнит пароль и не может войти. Сервер не присылает пароль и не даёт сменить его «по ссылке»: он проводит человека через три шага, и на каждом выдаёт новое подтверждение — сначала код из почты, потом короткоживущий «пропуск» на смену пароля.

Что нужно знать заранее

Все три запроса этого раздела **публичные**: человек ещё не вошёл, поэтому заголовка `Authorization` в них нет. Кто он такой, сервер выясняет по цепочке: **email** → **код из письма** → **resetToken**. Данные между страницами передаются только «в памяти» браузера (не в `localStorage`), поэтому после перезагрузки страницы процесс начинается заново.

Шаг 1 — вводим email FRONTEND

[← Back](#)

Forgot Password

We'll email a 6 digit code to your email address.

Email Address

shipinomaksim@gmail.com

Send OTP

Страница «Forgot Password»: только email. Кнопка «Send OTP» ещё не нажата — в Network пока пусто.

POST

http://localhost:4000/auth/forgot-password

200 OK

→ Payload — что браузер отправил на сервер (тело запроса)

Elements

Console

Sources

Network

Performance

Memory

Application

Security

>>

1

⚙

⋮

⏏

🚫

🔍

🔍

☐ Keep log

☐ Disable cache

No throttling

📶

⬆

⬇

⚙

Filter

☐ Invert

More filters

All

Fetch/XHR

Doc

CSS

JS

Font

Img

Media

Manifest

Socket

Wasm

Other

Name

✕

Headers

Payload

Preview

Response

Initiator

Timing

🔗 forgot-password

▼ Request payload

View source

▼ {email: "shipinomaksim@gmail.com"}
email: "shipinomaksim@gmail.com"

POST

http://localhost:4000/auth/forgot-password

200 OK

← Preview — что сервер ответил

Elements

Console

Sources

Network

Performance

Memory

Application

Security

>>

1

⚙

⋮

⏏

🚫

🔍

🔍

☐ Keep log

☐ Disable cache

No throttling

📶

⬆

⬇

⚙

Filter

☐ Invert

More filters

All

Fetch/XHR

Doc

CSS

JS

Font

Img

Media

Manifest

Socket

Wasm

Other

Name

✕

Headers

Payload

Preview

Response

Initiator

Timing

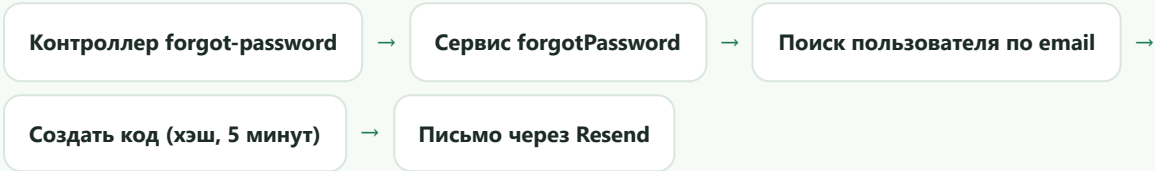
🔗 forgot-password

▼ {success: true, message: "Otp send to your email successfully",...}

▼ data: {createdAt: "2026-09-29T15:29:00.518Z"}
createdAt: "2026-09-29T15:29:00.518Z"
message: "Otp send to your email successfully"
success: true

Что улетело на сервер и что вернулось

Payload — одно поле: `email`. **Preview** — `success: true`, сообщение и `data.createdAt` — время, когда создан код. Самого кода в ответе нет: он приходит только на почту. По `createdAt` страница ввода кода рисует обратный отсчёт «OTP will expire in 04:58».



Простыми словами. Сервер ищет аккаунт с таким email. Если находит — придумывает шестизначный код, кладёт в базу только его хэш (в таблицу `password_reset_otps`, срок жизни 5 минут, счётчик неверных попыток = 0) и отправляет код на почту. Если такого email нет — не делает ничего, но отвечает **точно так же** «код отправлен». Так посторонний не может выяснить, кто зарегистрирован в системе.

```
// backend/src/auth/auth.service.ts

async forgotPassword(email: string) {
  const user = await this.prisma.user.findUnique({ where: { email } });
  let createdAt = new Date();

  if (user) {
    const generateotp = randomInt(100000, 1000000).toString();
    const otpHash = await bcrypt.hash(generateotp, 10);
    const expiredAt = new Date(Date.now() + 5 * 60 * 1000);

    const otp = await this.prisma.passwordResetOtp.create({
      data: { userId: user.id, codeHash: otpHash, expiresAt: expiredAt },
    });
    createdAt = otp.createdAt;

    await this.emailService.sendOtpForgotPassword(
      user.fullName, user.email, generateotp, '5 Min',
    );
  }

  // Одинаковый ответ и для существующего, и для несуществующего email.
  return {
    success: true,
    message: 'Otp send to your email successfully',
    data: { createdAt },
  };
}
```

Почему ответ одинаковый для любого email?

Если бы сервер отвечал «такого пользователя нет», любой мог бы перебором проверять, чьи почтовые ящики зарегистрированы. Честная оговорка: время ответа всё же чуть разное (для настоящего аккаунта сервер считает хэш и отправляет письмо), полностью выровнять его пока не сделано.

Шаг 2 — вводим код из письма

FRONTEND

← Back

Enter Verification otp

We sent a 6 digit otp to shipinomaksim@gmail.com.

OTP will expire in 04:55

Verify OTP

Страница ввода кода: идёт обратный отсчёт (5 минут). По нулям появляется «Resend Otp».

← Back

Enter Verification otp

We sent a 6 digit otp to shipinomaksim@gmail.com.

1

3

4

2

9

4

OTP will expire in 04:52

Verify OTP

Шесть цифр из письма введены, готовы нажать «Verify OTP».

POST

http://localhost:4000/auth/forgot-password/verify-otp

200 OK

→ Payload — что браузер отправил на сервер (тело запроса)

Elements

Console

Sources

Network

Performance

Memory

Application

Security

>>

1

⚙

⋮

⏻

🔍

🔍

🔍

Keep log

Disable cache

No throttling

📶

⬆

⬇

⚙

Filter

All

Fetch/XHR

Doc

CSS

JS

Font

Img

Media

Manifest

Socket

Wasm

Other

Name

✕

Headers

Payload

Preview

Response

Initiator

Timing

✖ forgot-password

✖ verify-otp

▼ Request payload

View source

▼ {email: "shipinomaksim@gmail.com", otp: "134294"}
email: "shipinomaksim@gmail.com"
otp: "134294"

POST

http://localhost:4000/auth/forgot-password/verify-otp

200 OK

← Preview — что сервер ответил (раскрыто поле data)

Elements

Console

Sources

Network

Performance

Memory

Application

Security

>>

1

⚙

⋮

⏻

🔍

🔍

🔍

Keep log

Disable cache

No throttling

📶

⬆

⬇

⚙

Filter

All

Fetch/XHR

Doc

CSS

JS

Font

Img

Media

Manifest

Socket

Wasm

Other

Name

✕

Headers

Payload

Preview

Response

Initiator

Timing

✖ forgot-password

✖ verify-otp

▼ {success: true, message: "Otp verify successfully", data: {,...}}

▼ data: {,...}

resetToken: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJjbXU1cXZtNGUwMDA"

message: "Otp verify successfully"

success: true

Что улетело на сервер и что вернулось

Payload — два поля: `email` и `otp` (код из письма). **Preview** — внутри `data` лежит `resetToken` : длинная подписанная строка, «пропуск» на смену пароля. Он живёт 5 минут и подписан отдельным секретом (`JWT_RESET_SECRET`), поэтому не подойдёт ни как токен доступа, ни как код 2FA. Внутри токена записана «версия пароля» — время его последней смены: по ней сервер потом поймёт, что токен уже использован.

Простыми словами. Сервер берёт самый свежий не использованный и не просроченный код этого пользователя. Если неверных попыток уже 5 — отказывает. Сверяет введённый код с хэшем; при ошибке прибавляет единицу к счётчику попыток. Если всё верно — помечает код «использованным» (второй раз он не сработает) и выдаёт `resetToken`.

```
// backend/src/auth/auth.service.ts
```

```
async forgotPasswordOtpVerification(email: string, otp: string) {
  const user = await this.prisma.user.findUnique({ where: { email } });
  if (!user) throw new BadRequestException('Otp does not match');

  const passwordResetOtp = await this.prisma.passwordResetOtp.findFirst({
    where: { userId: user.id, consumedAt: null, expiresAt: { gt: new Date() } },
    orderBy: { createdAt: 'desc' },
  });
  if (!passwordResetOtp) throw new BadRequestException('Otp expired or invalid');
  if (passwordResetOtp.attempts >= 5) {
    throw new BadRequestException('Too many otp attempts');
  }

  const ok = await bcrypt.compare(otp, passwordResetOtp.codeHash);
  if (!ok) {
    await this.prisma.passwordResetOtp.update({
      where: { id: passwordResetOtp.id },
      data: { attempts: { increment: 1 } },
    });
    throw new BadRequestException('Otp does not match');
  }

  // Код «сгорает» после первой удачной проверки.
  await this.prisma.passwordResetOtp.update({
    where: { id: passwordResetOtp.id },
    data: { consumedAt: new Date() },
  });

  // В токен кладем «версию пароля»: она понадобится, чтобы токен сработал лишь один раз.
  const resetToken = await this.jwtService.signAsync(
    { userId: user.id, passwordVersion: user.passwordUpdatedAt.getTime() },
    { secret: this.config.getOrThrow('JWT_RESET_SECRET'), expiresIn: '5m' },
  );
  return { success: true, message: 'Otp verify successfully', data: { resetToken } };
}
```

FRONTEND

Already had an account: [Sign in](#)

Already had an account: [Sign in](#)

Пароль и подтверждение введены. Совпадение проверяет только браузер — на сервер оно не уходит.

The screenshot shows the Chrome DevTools Network tab. The 'reset-password' request is selected. The 'Preview' pane shows the following JSON response:

```
{
  "user": {
    "id": "cmu5qvm4e0000jcv6j849ze2i",
    "email": "shipinomaksim@gmail.com",
    "phone": "380934567890"
  },
  "message": "Password changed successfully",
  "success": true
}
```



Password Updated

Your password has been reset. You can now sign in with your new password.

[Back to sign in](#)

Готово: «Password Updated». Ссылка ведёт обратно на страницу входа.

Что улетело на сервер и что вернулось

Payload — два поля: новый `password` и `resetToken` из предыдущего шага. Ни email, ни старого пароля тут нет — личность подтверждает сам токен. **Preview** — `success: true`, сообщение «Password changed successfully» и профиль пользователя (без хэша пароля и секрета 2FA).

```
// backend/src/users/users.service.ts

async resetForgottenPassword(userId: string, dto: ResetForgottenPasswordDto,
    expectedPasswordVersion: number) {
    const passwordHash = await bcrypt.hash(dto.password, 10);

    // Всё в одной транзакции: либо и пароль сменился, и сессии закрылись, либо ничего.
    const user = await this.prisma.$transaction(async (tx) => {
        // Пароль меняется, только если он не менялся с момента выдачи токена.
        // Повторное использование токена не найдёт ни одной строки.
        const claimed = await tx.user.updateMany({
            where: { id: userId, passwordUpdatedAt: new Date(expectedPasswordVersion) },
            data: { passwordHash, passwordUpdatedAt: new Date() },
        });
        if (claimed.count === 0) throw new BadRequestException('Session is Expired');

        // При сбросе никто не «залогинен», поэтому закрываем ВСЕ живые сессии.
        await tx.userSession.updateMany({
            where: { userId, revokedAt: null },
            data: { revokedAt: new Date() },
        });
        return tx.user.findUniqueOrThrow({ where: { id: userId },
            omit: { passwordHash: true, twoFactorSecret: true } });
    });

    return { user, message: 'Password changed successfully', success: true };
}
```

Простыми словами. Сервер проверяет подпись и срок жизни `resetToken` (если токен подделан или просрочен — аккуратный ответ 400 «Session is Expired»). Затем меняет пароль **только если он не менялся с момента выдачи токена**: после первого успешного сброса «версия пароля» меняется, и тот же токен второй раз уже не подойдёт. В той же операции «отзываются» все сессии этого человека: если кто-то сидел в аккаунте с украденным токеном, его выкинет.

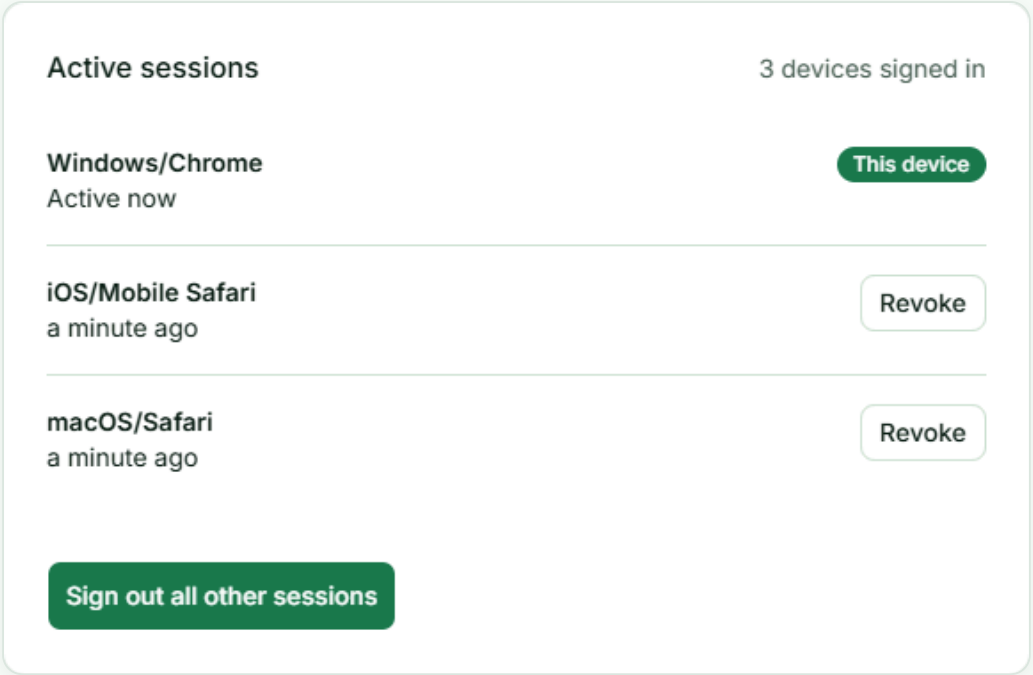
Что усилено в этом сценарии

1) `resetToken` одноразовый: он привязан к версии пароля, повтор даёт 400. **2)** Просроченный или подделанный токен — это аккуратный 400, а не ошибка сервера. **3)** Длина нового пароля (8–72 символа) проверяется и на этом адресе, как при обычной смене пароля. **Остаётся:** сброс не требует 2FA — кто владеет почтой, тот обходит второй фактор; это решение о политике безопасности, а не ошибка кода. Подробности — в `AUTH_FLOW.md`, раздел 14a.

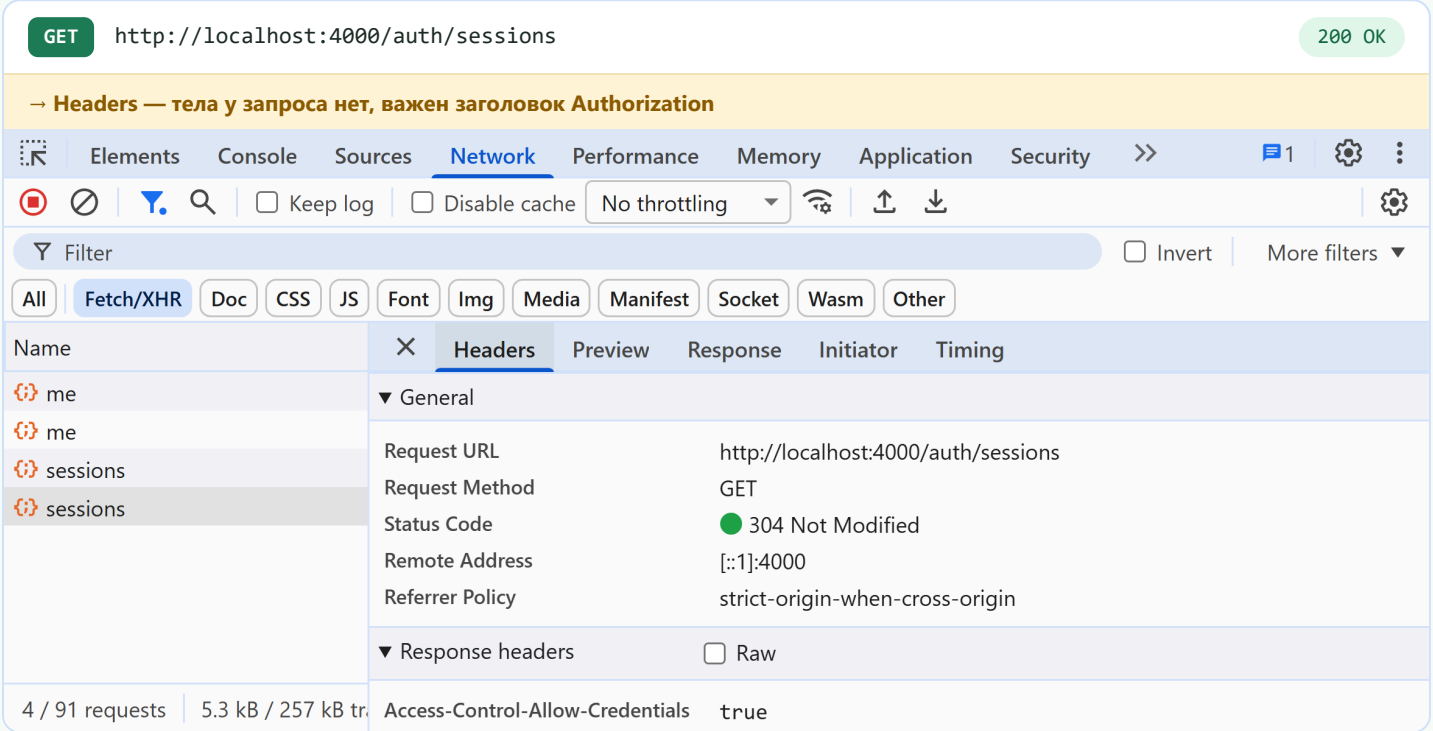
8 Активные сессии и выход с устройств

Каждый вход создаёт в базе запись о сессии: с какого устройства, из какой системы и браузера, когда была последняя активность. В разделе «Мой аккаунт» человек видит этот список и может закрыть любую чужую или забытую сессию.

Шаг 1 — открываем «Мой аккаунт» FRONTEND



Три сессии: эта (Windows/Chrome, помечена «This device») и две другие. Названия строятся из разобранного заголовка User-Agent: система и браузер.



GET

http://localhost:4000/auth/sessions

200 OK

Preview — что сервер ответил (раскрыт первый элемент списка)

Elements

Console

Sources

Network

Performance

Memory

Application

Security

>>

1

⚙

⋮

⏏

🚫

🔍

☐ Keep log

☐ Disable cache

No throttling

📶

⬆

⬇

⚙

🔍 Filter

☐ Invert

More filters ▾

All

Fetch/XHR

Doc

CSS

JS

Font

Img

Media

Manifest

Socket

Wasm

Other

Name

me

me

sessions

sessions

×

Headers

Preview

Response

Initiator

Timing

[{deviceLabel: null, ipAddress: "::ffff:127.0.0.1", lastActiveAt: "2026-09-29T15:38:24.195Z", ...}]

▼ 0: {deviceLabel: null, ipAddress: "::ffff:127.0.0.1", lastActiveAt: "2026-09-29T15:38:24.195Z", ...}

browser: "Mobile Safari"

createdAt: "2026-09-29T15:38:24.195Z"

device: "iPhone"

deviceLabel: null

expiresAt: "2026-10-29T15:38:24.000Z"

ipAddress: "::ffff:127.0.0.1"

isCurrent: false

lastActiveAt: "2026-09-29T15:38:24.195Z"

os: "iOS"

sessionId: "rfleqfy3b35cssigs8fpiy4n"

4 / 92 requests

4.0 kB / 19.1 kB transferred

Что вернулось

Массив сессий. У каждой: `os`, `browser`, `device` (модель устройства, у компьютеров пусто), `ipAddress`, `createdAt`, `lastActiveAt`, `expiresAt`, идентификатор `sessionId` и флаг `isCurrent`: сервер сравнивает id каждой записи с id сессии из вашего токена, и текущая получает бейдж «This device» без кнопки «Revoke».

● ● ●

TYPESCRIPT

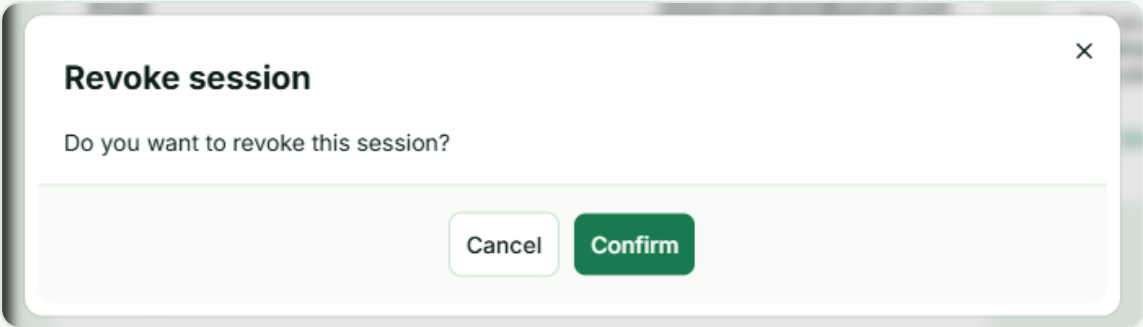
```
// backend/src/auth/auth.service.ts

async listSessions(userId: string) {
  return this.prisma.userSession.findMany({
    where: { userId, revokedAt: null }, // только не отозванные
    orderBy: { lastActiveAt: 'desc' }, // самые свежие сверху
  });
}

async revokeSession(userId: string, sessionId: string) {
  const session = await this.prisma.userSession.findUnique({ where: { id: sessionId } });
  // Чужую сессию закрыть нельзя: id должен принадлежать вам.
  if (!session || session.userId !== userId) {
    throw new BadRequestException('Session not found');
  }
  await this.prisma.userSession.update({
    where: { id: sessionId },
    data: { revokedAt: new Date() },
  });
}

async revokeOtherSessions(userId: string, currentSessionId: string) {
  await this.prisma.userSession.updateMany({
    where: { userId, revokedAt: null, NOT: { id: currentSessionId } },
    data: { revokedAt: new Date() },
  });
}
```

Шаг 2 — закрываем одну сессию FRONTEND



Кнопка «Revoke» открывает подтверждение. У текущего устройства такой кнопки нет.

POST http://localhost:4000/auth/sessions/<id>/revoke 200 OK

→ Headers — адрес с id сессии; метод POST, тела нет

Elements Console Sources Network Performance Memory Application Security >> 1 ⚙ ⋮

⏹ Filter

All Fetch/XHR Doc CSS JS Font Img Media Manifest Socket Wasm Other

6 / 95 requests | 8.0 kB / 259 kB tr

⌵ Headers Preview Response Initiator Timing

me me sessions sessions revoke sessions

General

Request URL http://localhost:4000/auth/sessions/fcrujywhnlam8uhxj8umxtwn/revoke

Request Method POST

Status Code 200 OK

Remote Address [::1]:4000

Referrer Policy strict-origin-when-cross-origin

Response headers Raw

Access-Control-Allow-Credentials true

POST http://localhost:4000/auth/sessions/<id>/revoke 200 OK

← Preview — что сервер ответил

Elements Console Sources Network Performance Memory Application Security >> 1 ⚙ ⋮

⏹ Filter

All Fetch/XHR Doc CSS JS Font Img Media Manifest Socket Wasm Other

6 / 95 requests | 8.0 kB / 259 kB tr

⌵ Headers Preview Response Initiator Timing

me me sessions sessions

{success: true}

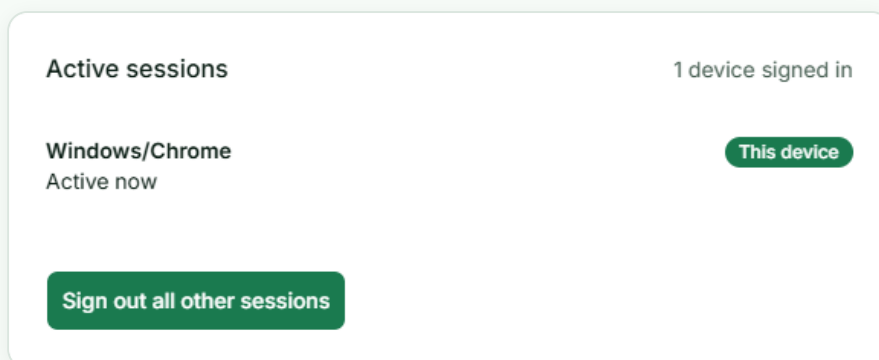
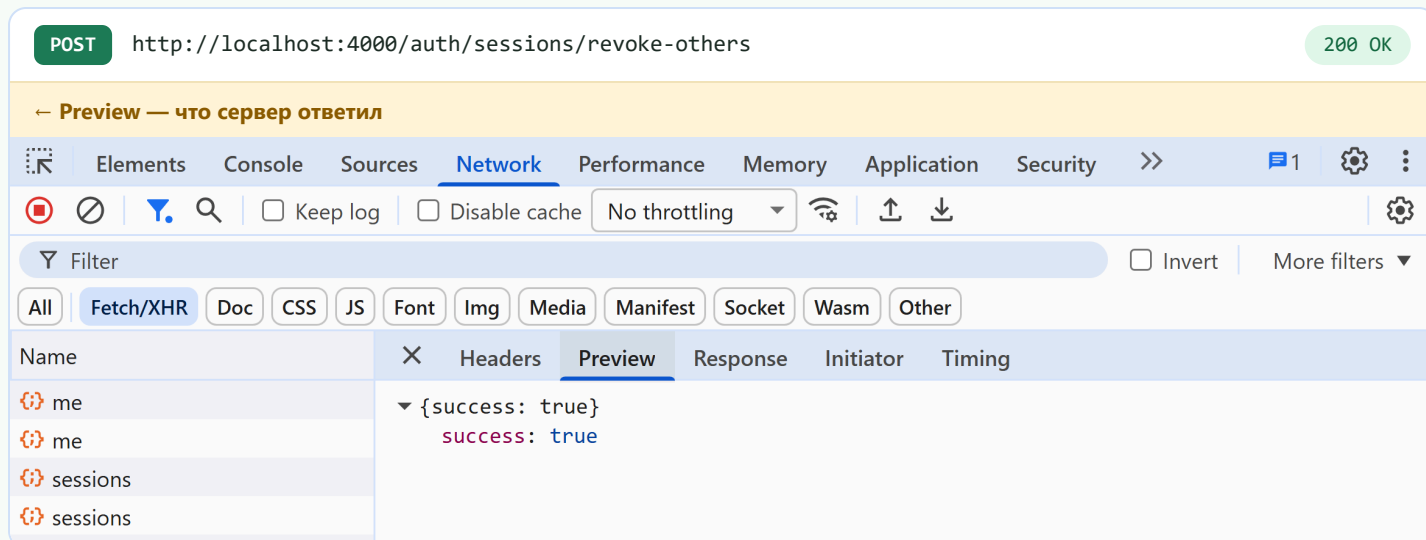
success: true

После подтверждения список перезагрузился: осталось две сессии.

Токен доступа сам по себе живёт ещё до 15 минут, но на каждом запросе охранник (`JwtStrategy`) заглядывает в запись сессии в базе. Стоит там появиться отметке «отозвана» — следующий запрос с этого устройства получит 401, а тихое обновление токена (`refresh`) тоже откажет.

FRONTEND

The screenshot shows the Chrome DevTools Network tab. A POST request to `http://localhost:4000/auth/sessions/revoke-others` is selected. The status is `200 OK`. The 'Headers' sub-tab is active, showing the 'General' section with details like Request URL, Request Method, Status Code, Remote Address, and Referrer Policy. The 'Response headers' section is expanded, showing `Access-Control-Allow-Credentials: true`. The breadcrumb at the top indicates the request is from a token.



Остался единственный вход — текущий.

Что значит «последняя активность»

Время `lastActiveAt` обновляется, когда браузер тихо получает новую пару токенов (`POST /auth/refresh`), а не при каждом клике. Поэтому оно двигается «рывками» — примерно раз в срок жизни токена доступа.

А обычный выход?

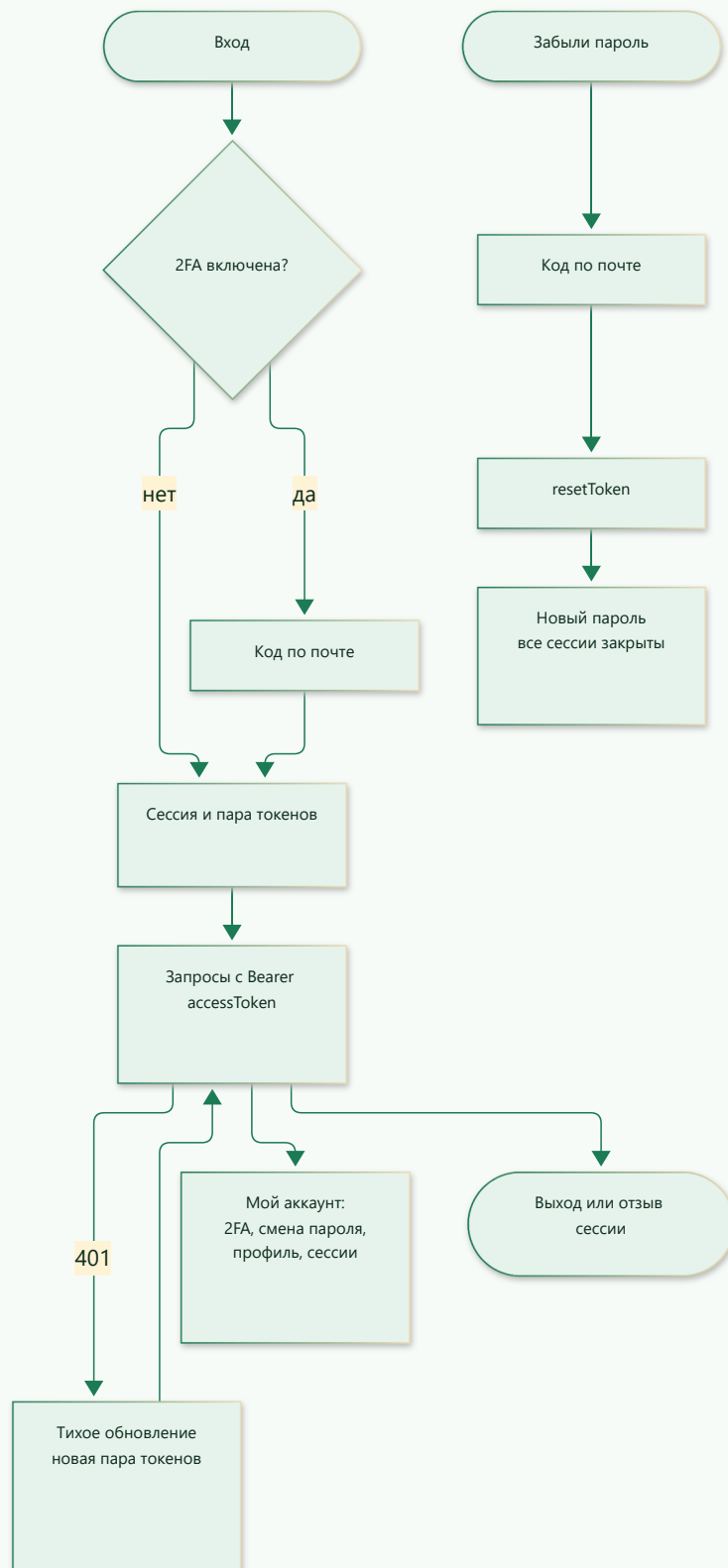
Кнопка выхода в боковом меню вызывает `POST /auth/logout` : закрывается только текущая сессия (запись помечается «отозвана»), остальные устройства остаются в системе. Повторный вызов безвреден.

9 Диаграммы последовательности

Те же сценарии, но в виде схем «кто кому что отправляет и в каком порядке». Стрелка — это сообщение или запрос, пунктир — ответ, вертикальные линии — участники. Схемы векторные: в PDF их можно увеличивать без потери чёткости.

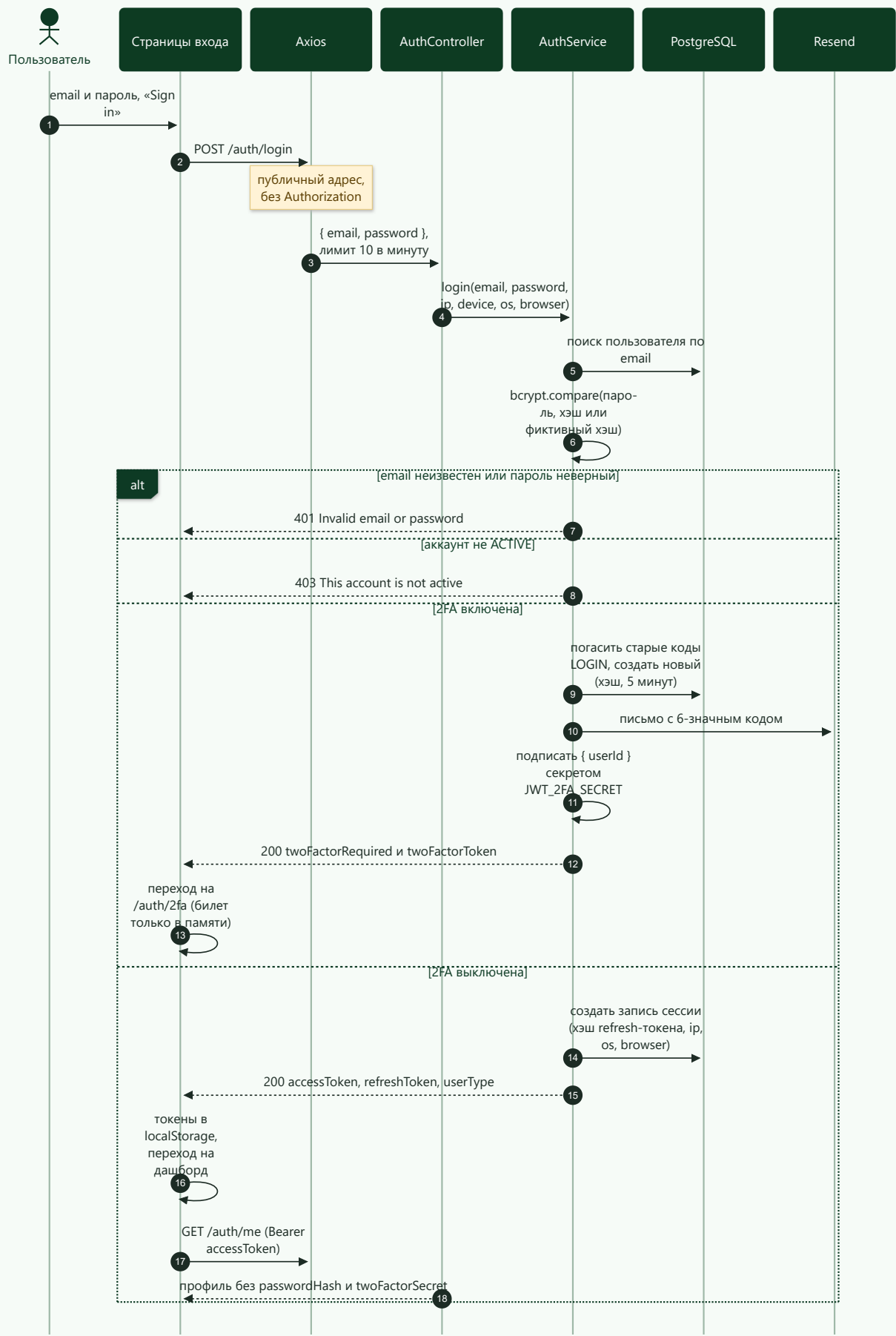
Общая картина: весь жизненный цикл

Одна схема, чтобы увидеть, как связаны все сценарии документа: вход, обновление токена, действия внутри аккаунта, выход и восстановление пароля.



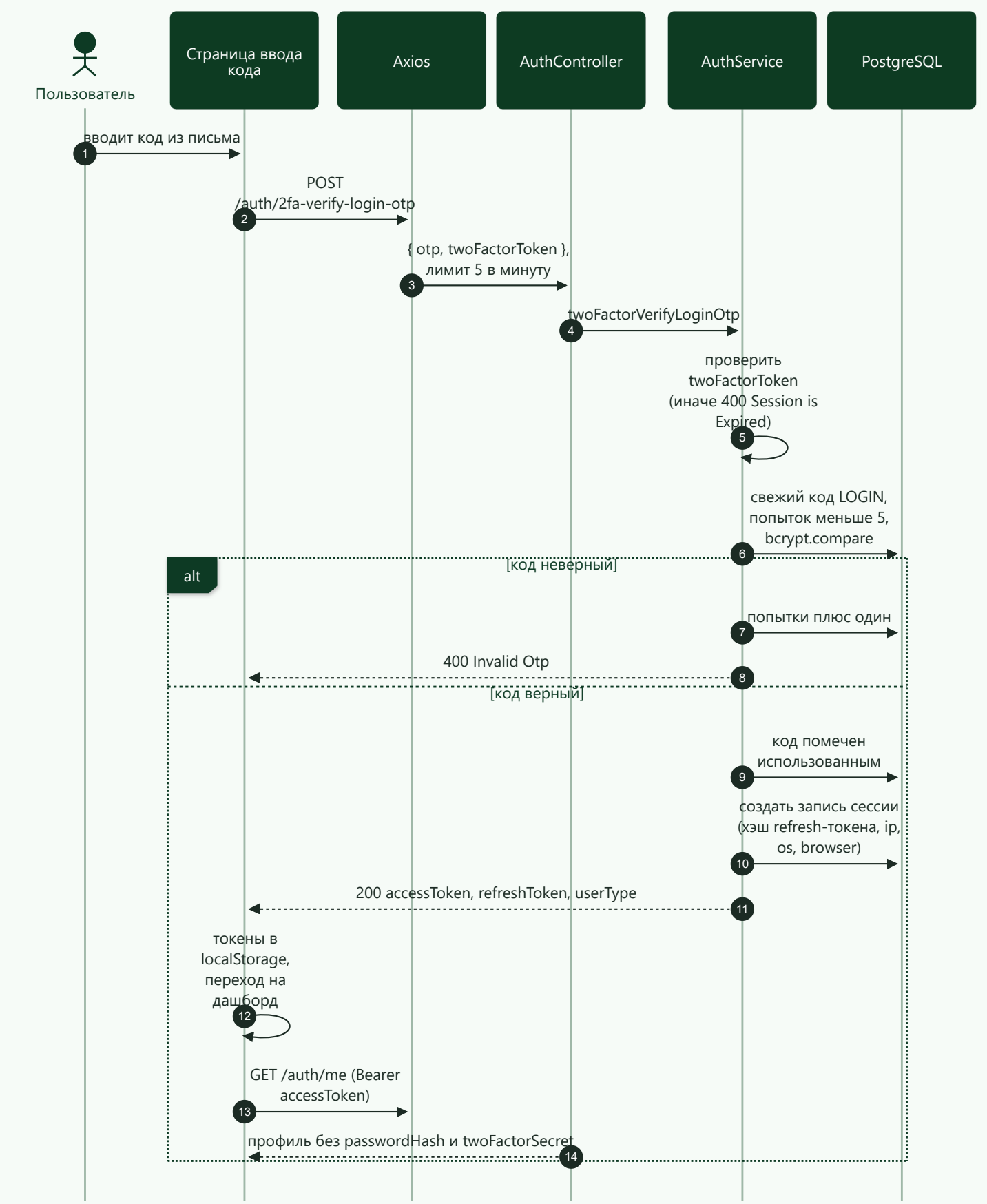
Вход в аккаунт, шаг 1: проверка пароля

Пароль проверяется всегда, даже для несуществующего email (по фиктивному хэшу, чтобы время ответа не выдавало ничего). Если 2FA включена, токенов ещё нет: сервер присылает «билет» twoFactorToken и отправляет код на почту.



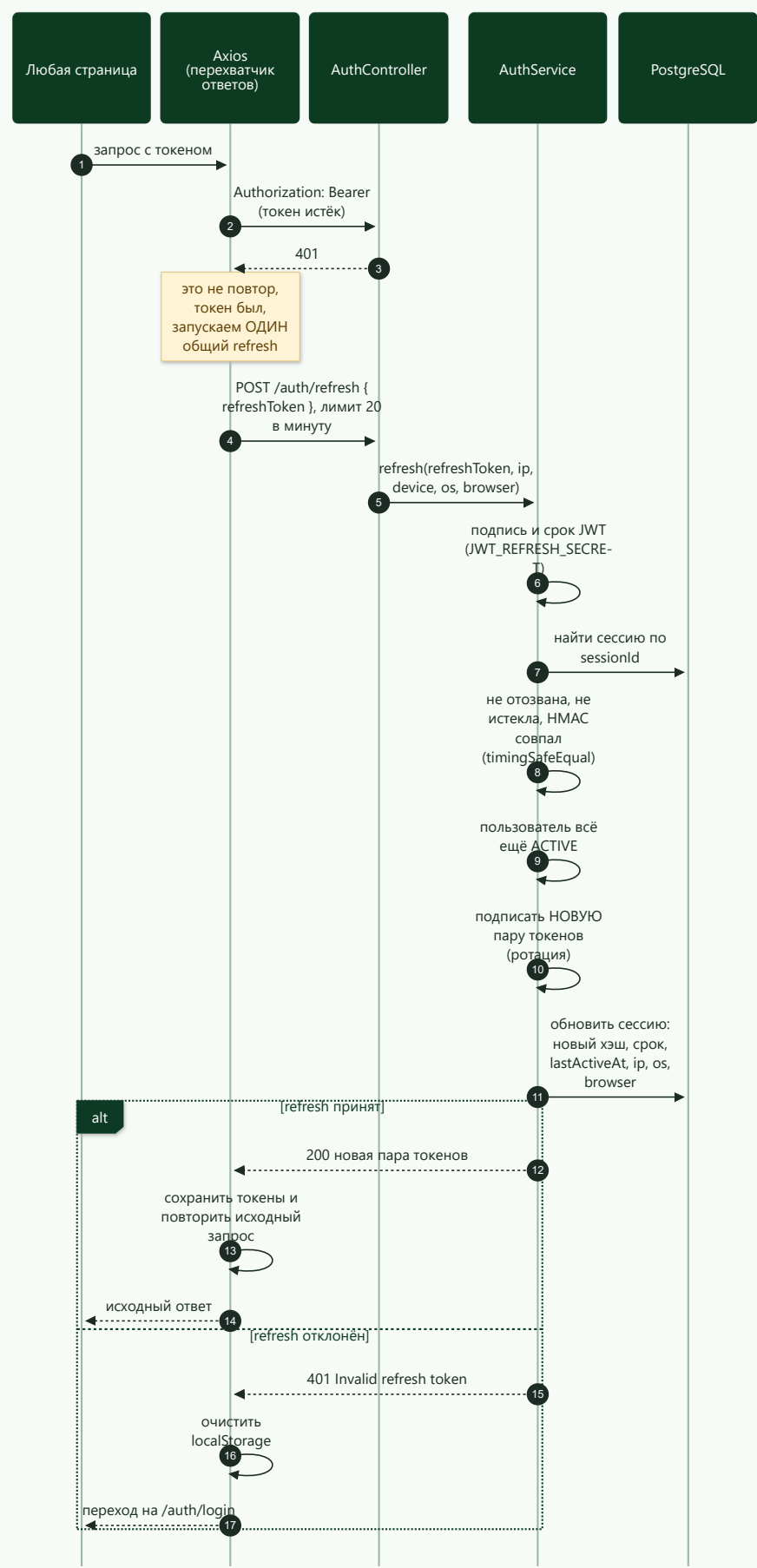
Вход в аккаунт, шаг 2: код из письма (только при 2FA)

Билет twoFactorToken подписан отдельным секретом, поэтому не годится как токен доступа. Сессия создаётся только после верного кода.



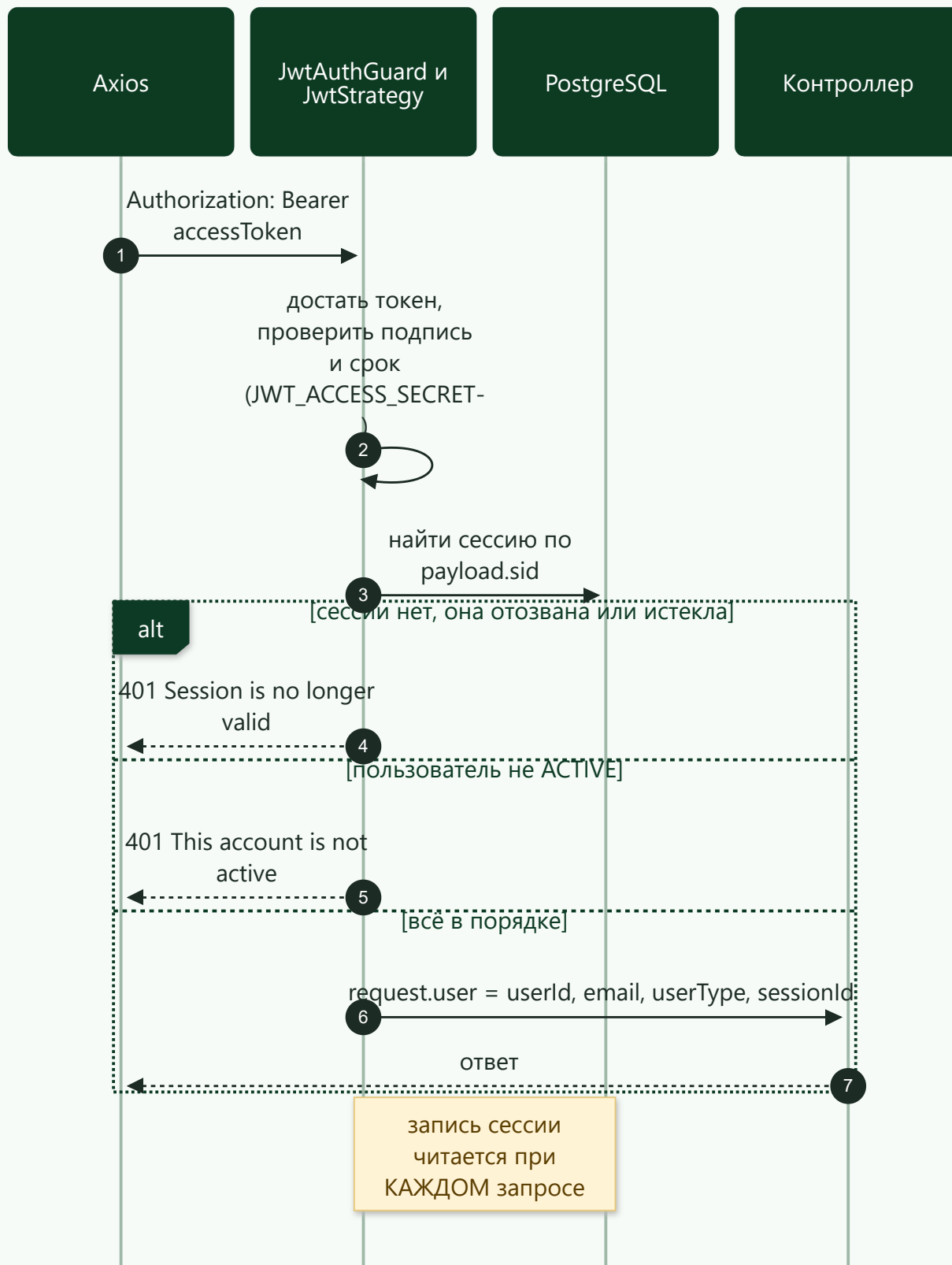
Тихое обновление токена

Когда токен доступа истёк, браузер сам, без участия человека, получает новую пару токенов. Параллельные запросы делят один общий refresh.



Что проверяет «охранник» на каждом запросе

Запись о сессии перечитывается из базы при каждом обращении, поэтому отозванная сессия перестаёт работать мгновенно, даже если токен технически ещё жив.

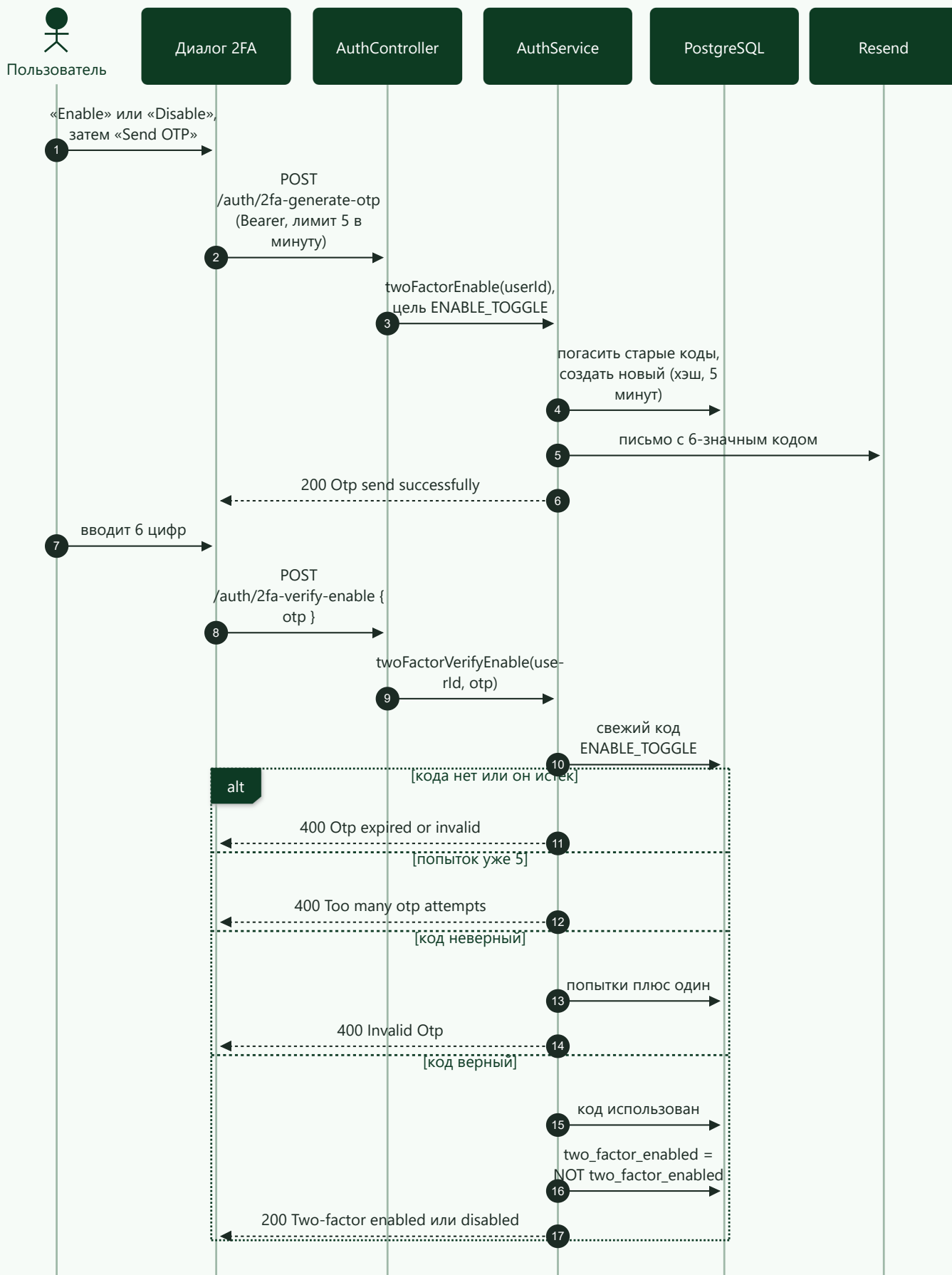


Кто есть кто на схемах

Пользователь — человек за браузером. **Axios** — код фронтенда, который отправляет запросы и сам обновляет токены. **AuthController** и **UsersController** — «приёмные» бэкенда. **AuthService** и **UsersService** — там живёт вся логика. **PostgreSQL** — база. **Resend** — сервис, отправляющий письма.

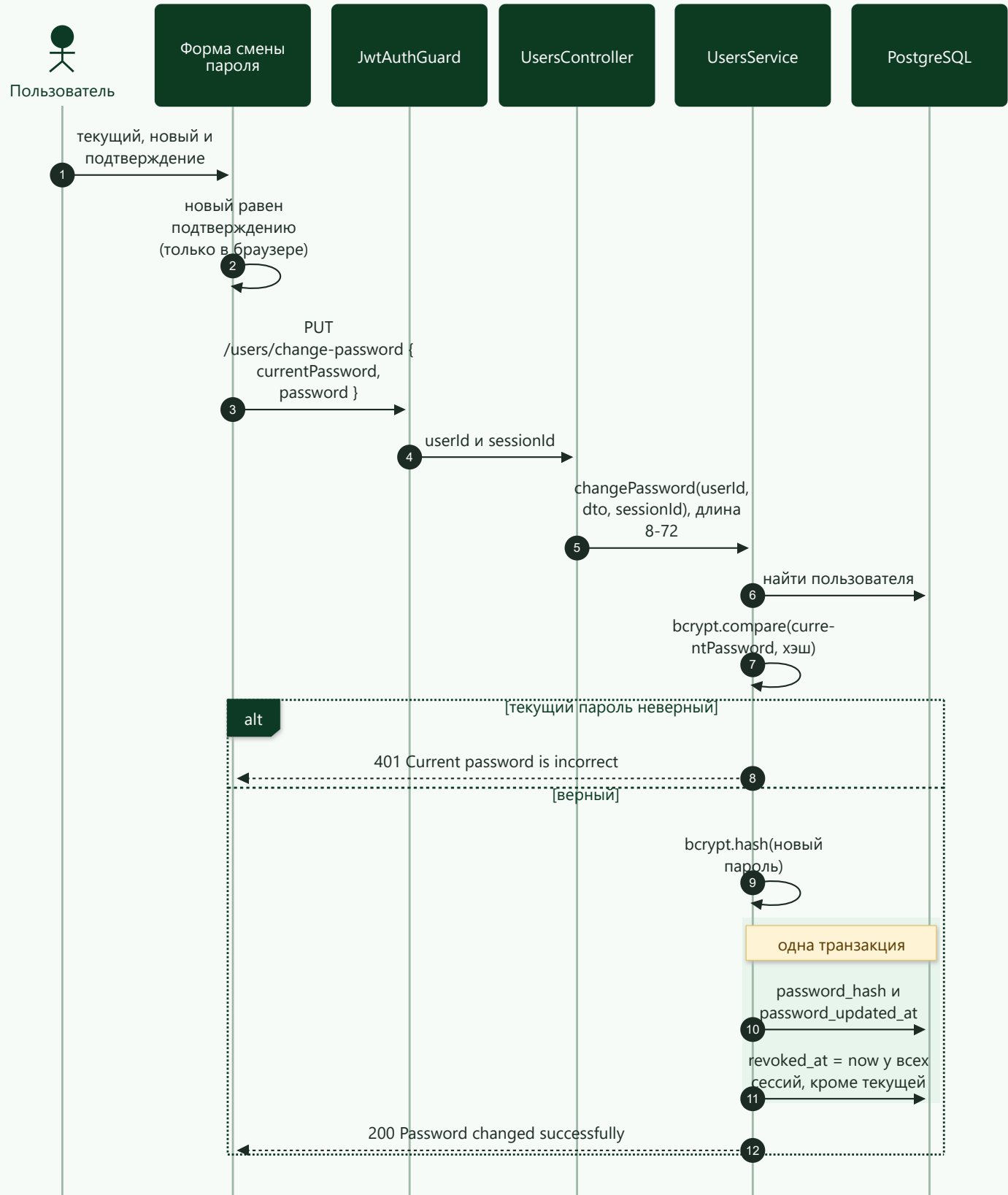
Двухфакторная аутентификация: включение и выключение

Один и тот же переключатель: сервер всегда ставит значение, противоположное текущему. Код привязан к цели ENABLE_TOGGLE и не подойдёт для входа.



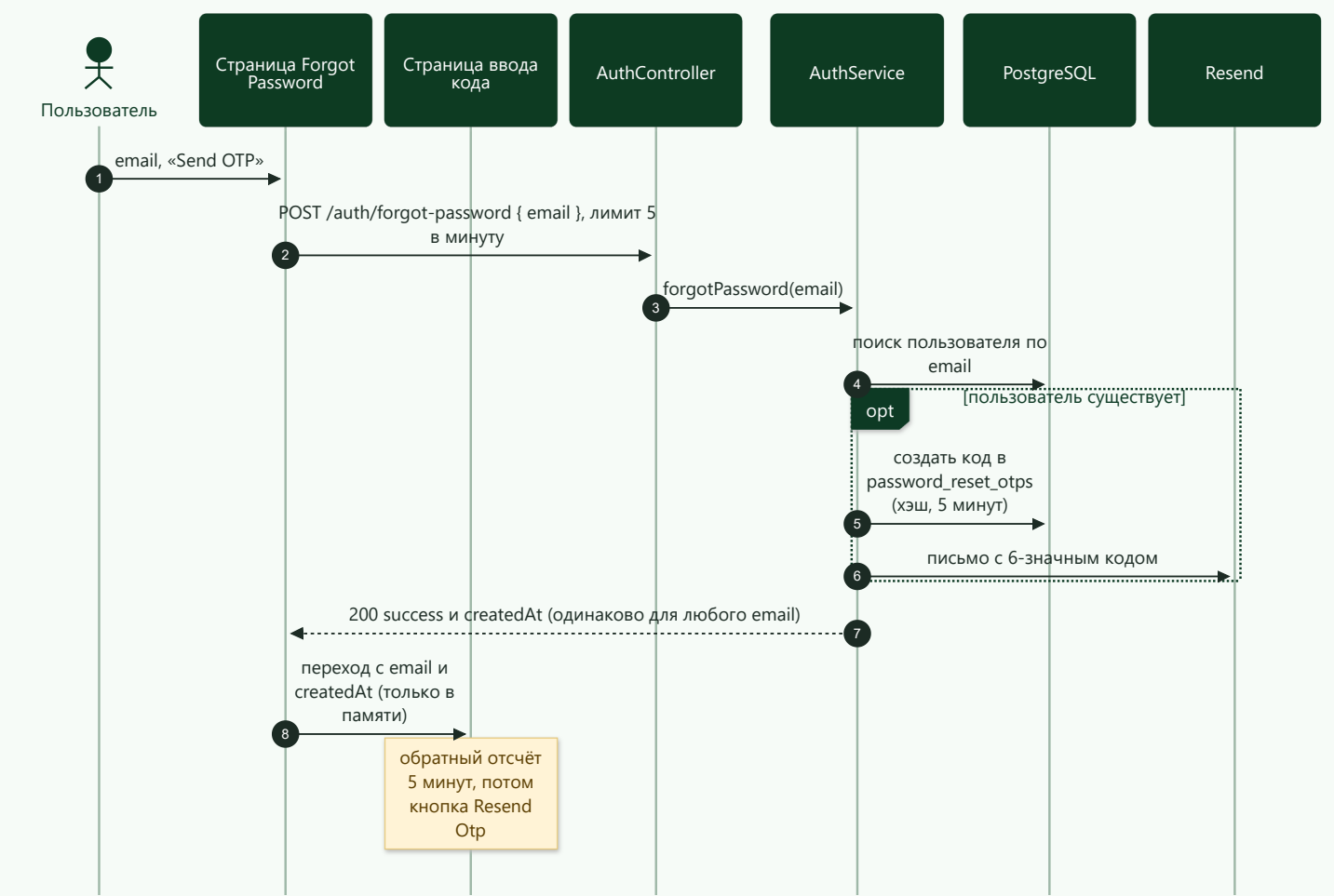
Смена пароля (когда человек вошёл)

Нужен текущий пароль. Новый хэш и отзыв остальных сессий записываются в одной транзакции, а текущая сессия остаётся живой.



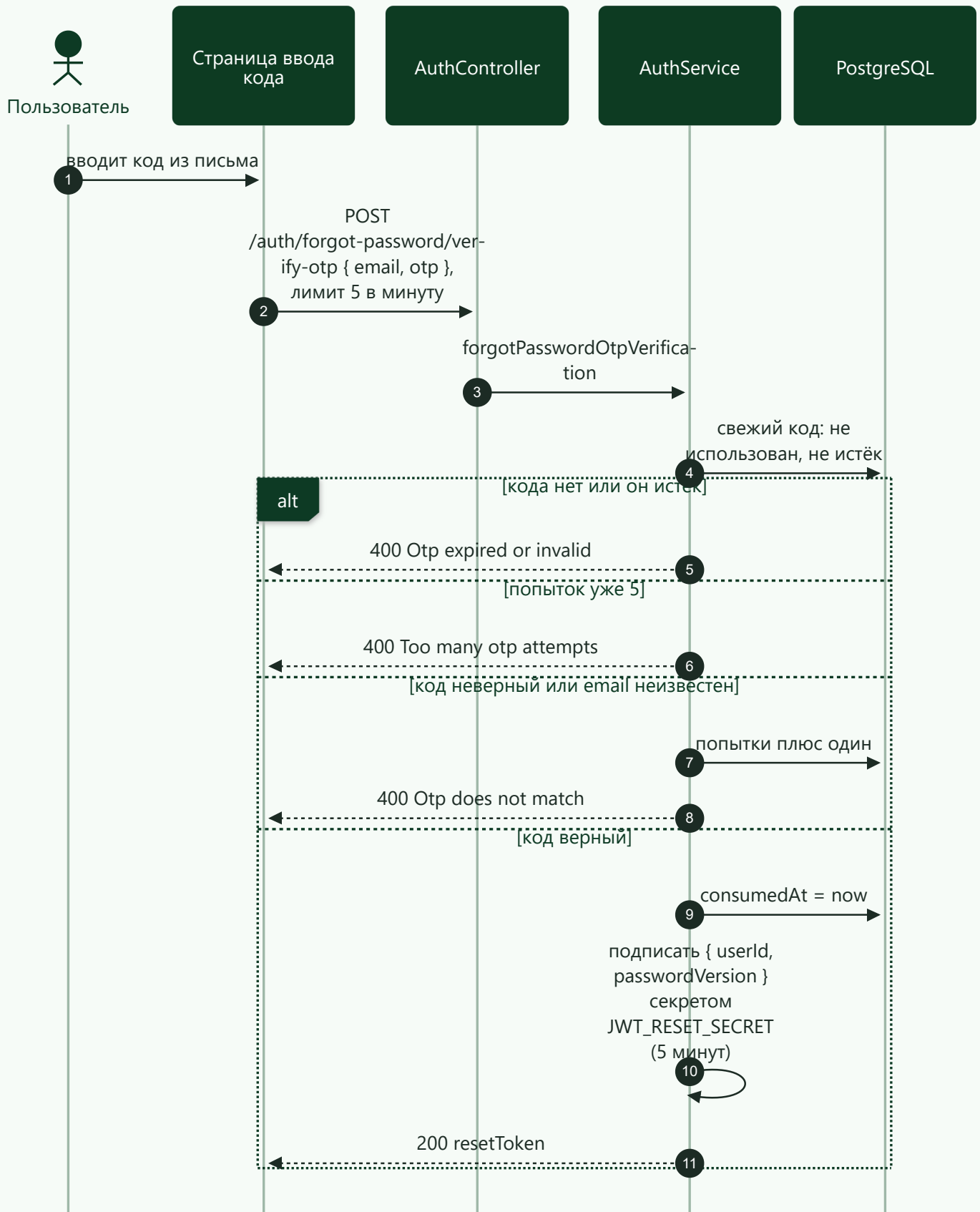
Забыли пароль, часть 1а: запрос кода

Ответ сервера одинаков для любого email, поэтому по нему нельзя узнать, кто зарегистрирован. В базе лежит только хэш кода.



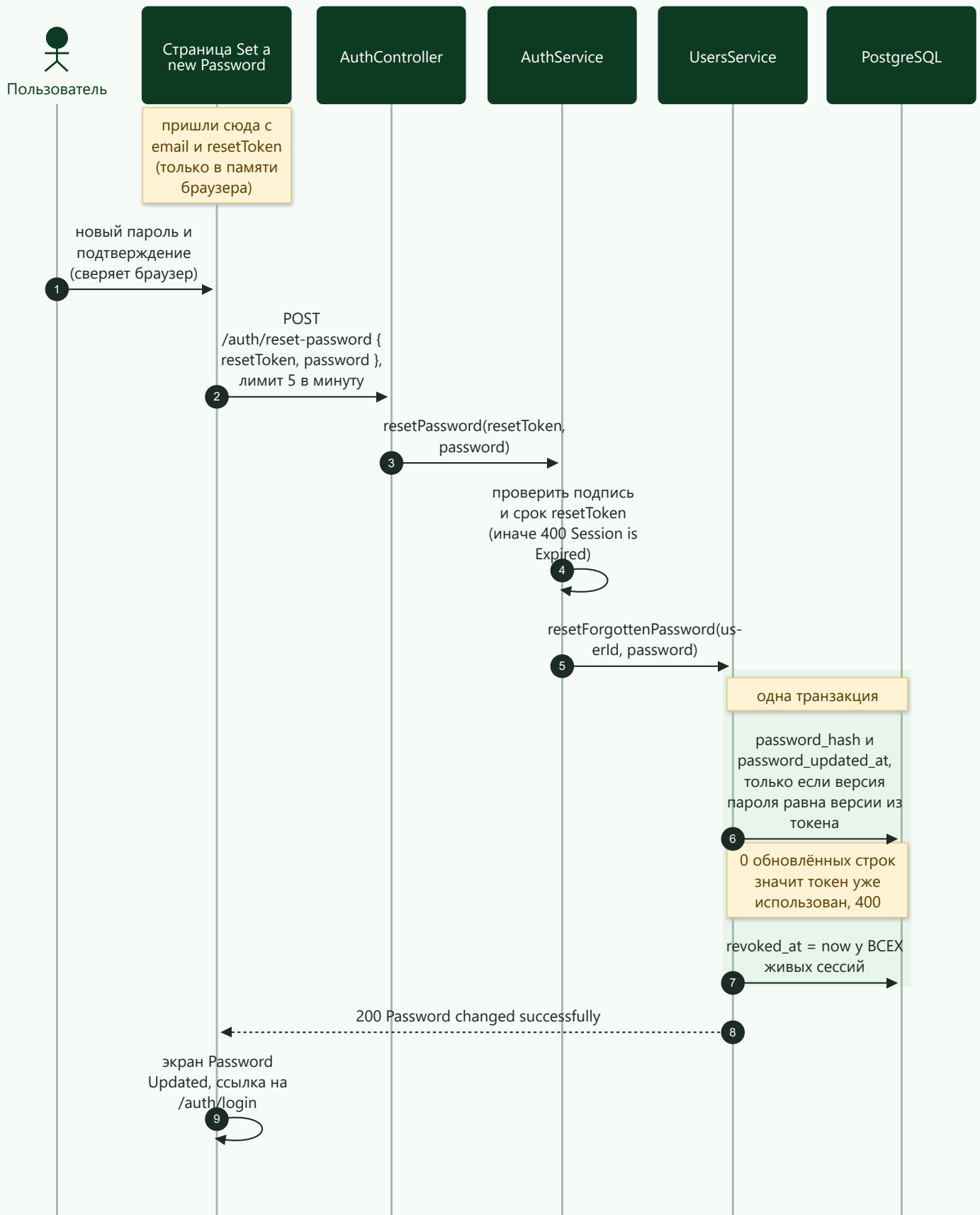
Забыли пароль, часть 16: проверка кода

Код одноразовый: после первой удачной проверки он помечается использованным. Неверные попытки считаются, после пятой код перестаёт приниматься.



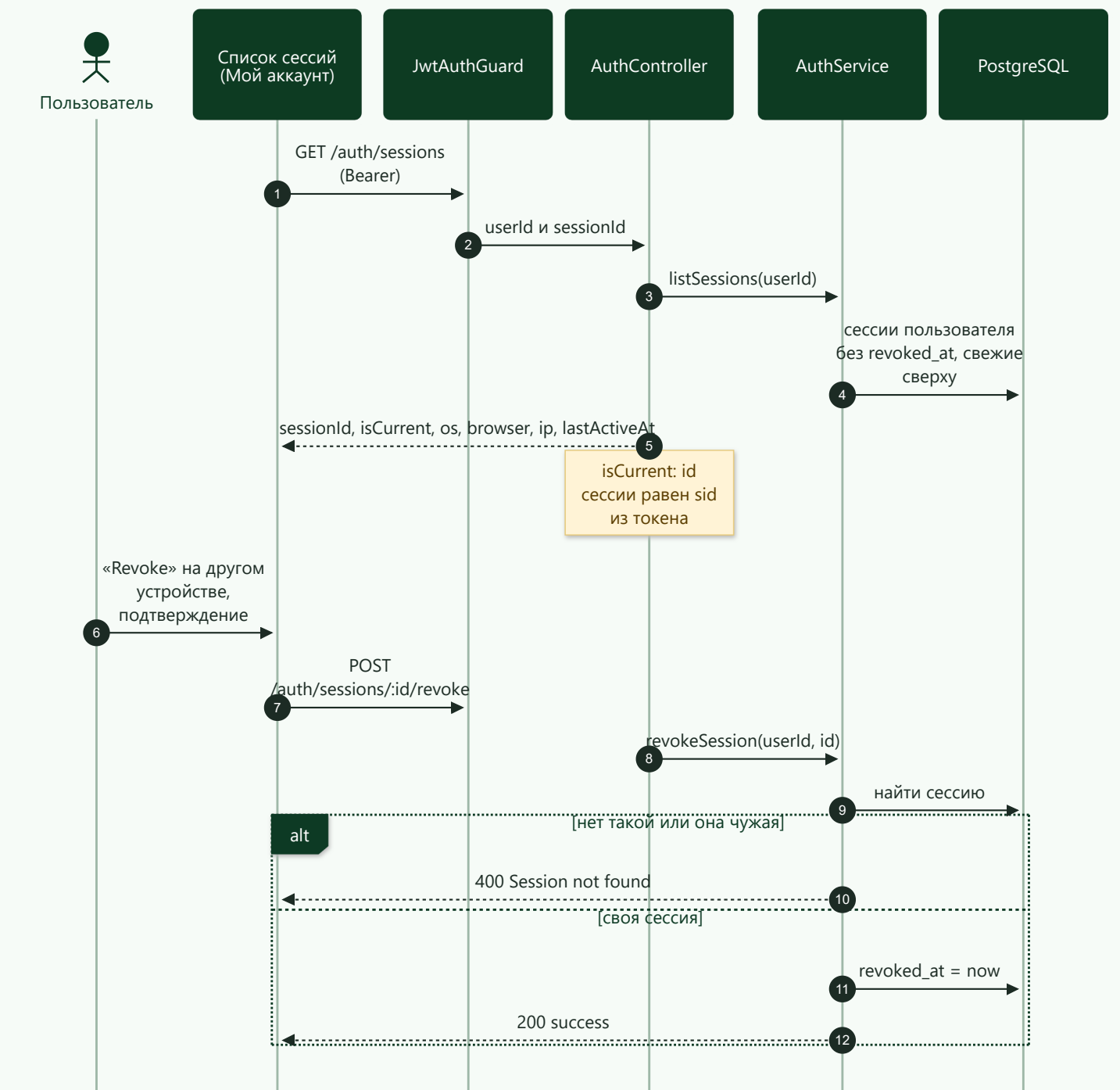
Забыли пароль, часть 2: новый пароль

resetToken подтверждает личность вместо старого пароля и срабатывает только один раз: он привязан к версии пароля. После сброса закрываются все сессии: в этот момент никто не вошёл, а украденный токен должен перестать работать.



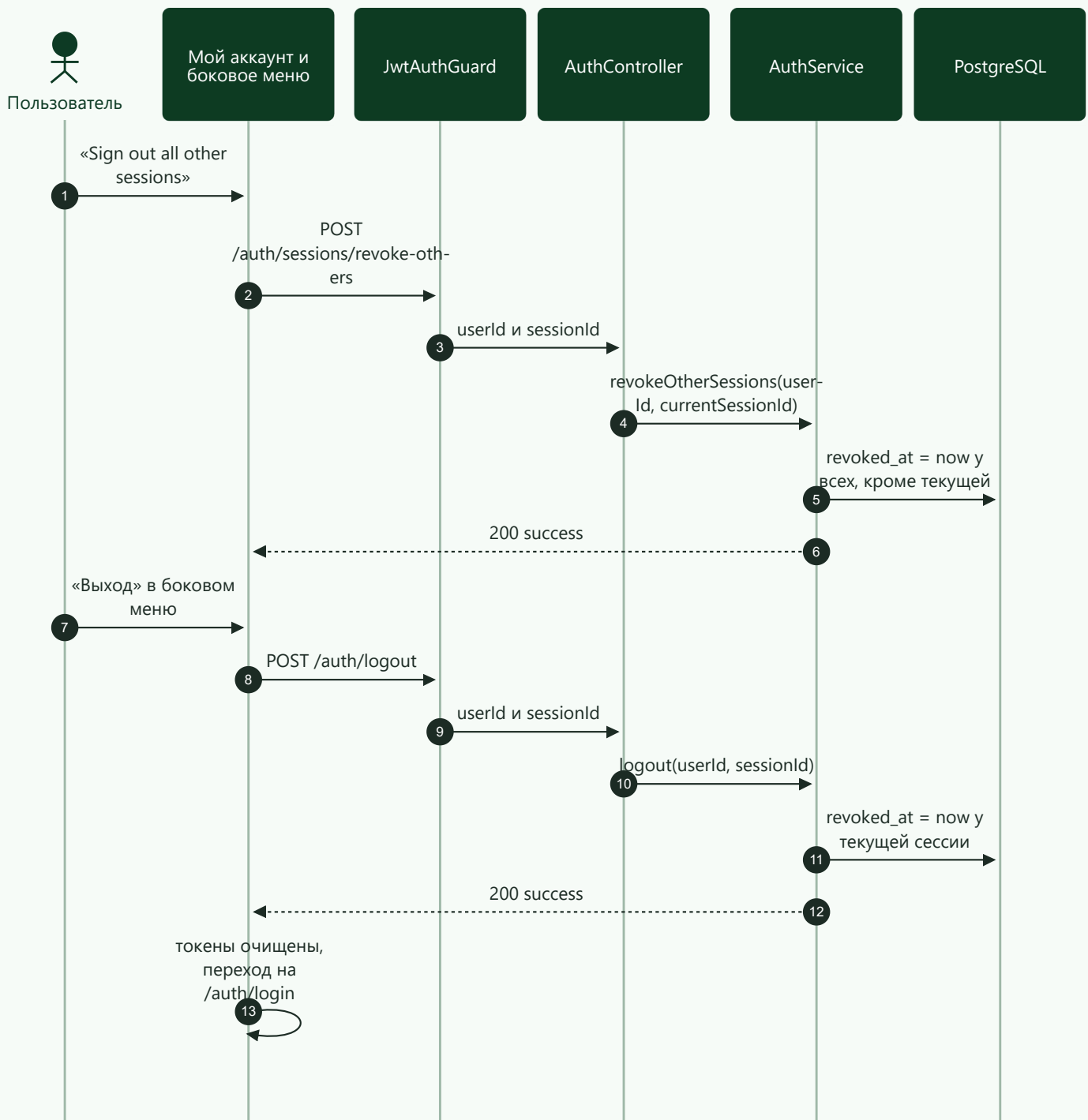
Активные сессии: список и отзыв одной

Список показывает только не отозванные сессии текущего пользователя. Чужую сессию закрыть нельзя: сервер сверяет владельца.



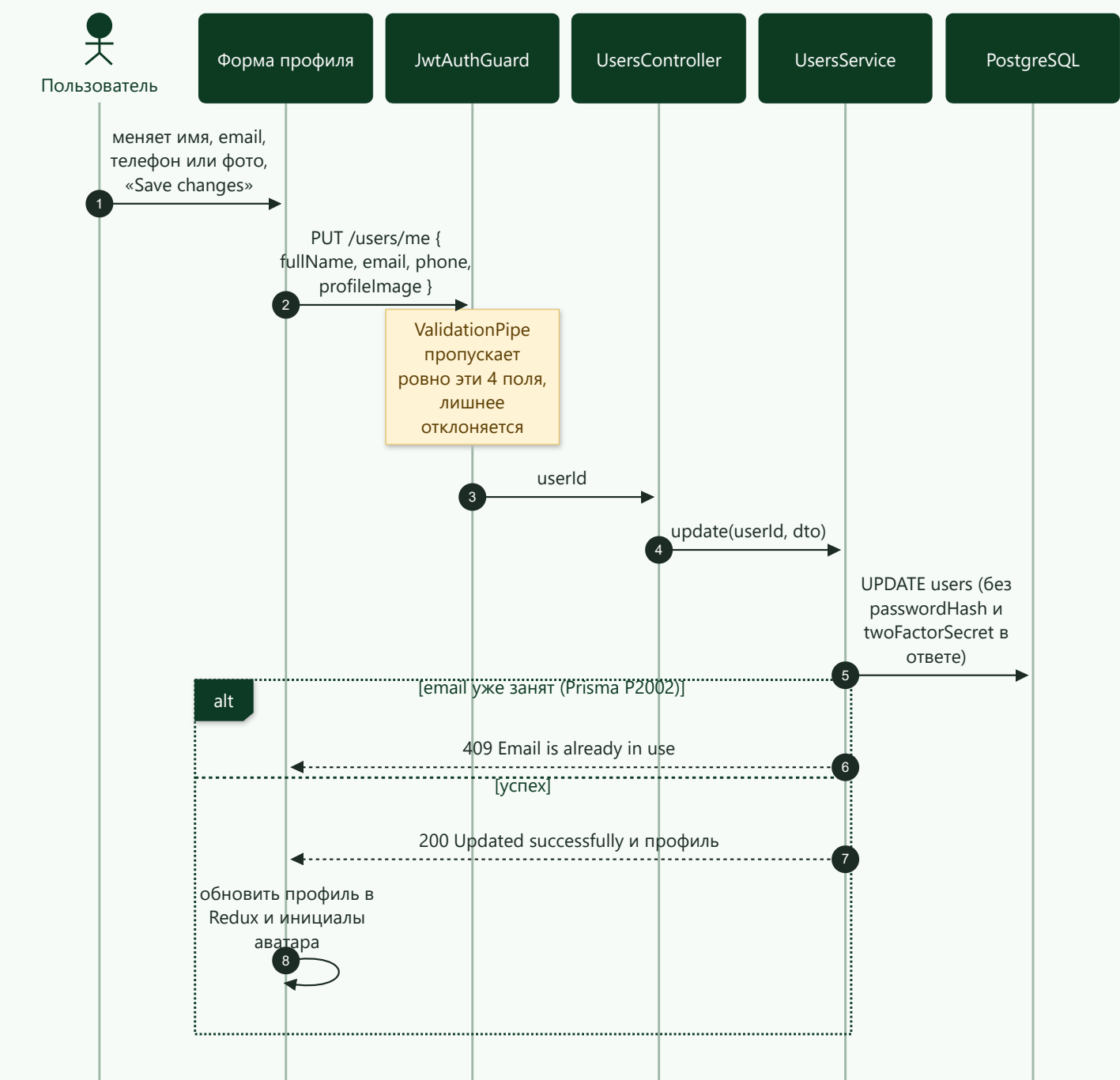
Активные сессии: выход со всех остальных и обычный выход

Обычный выход закрывает только текущую сессию. Кнопка «Sign out all other sessions» закрывает все, кроме текущей.



Редактирование профиля

Форма умеет менять только четыре поля. Поля password в этом запросе нет физически: пароль меняется только через отдельный адрес.



10 Что защищает аккаунт: итоги

Коротко, без кода — что происходит «под капотом» всегда, вне зависимости от конкретного экрана.

Защита	Что это значит простыми словами
Пароли и коды не хранятся в открытом виде	В базе лежит только необратимый «отпечаток» (хэш). Даже у разработчиков нет способа посмотреть исходный пароль или код.
Одинаковый ответ на неверный email и неверный пароль	Сообщение об ошибке при входе не выдаёт, зарегистрирован ли вообще такой email в системе. То же и при «Забыли пароль»: ответ одинаковый для любого адреса.
Ограничение попыток ввода кода	После 5 неверных попыток код перестаёт приниматься — перебором его не подобрать. Это касается и кодов 2FA, и кодов сброса пароля.
Ограничение частоты запросов (Rate limiting)	Чувствительные адреса (вход — 10 раз в минуту; коды 2FA, сброс пароля — по 5) не дадут постучаться чаще с одного места.
Разные секреты для разных токенов	Ключ доступа, ключ обновления сессии, ключ для кода 2FA и ключ для сброса пароля подписаны разными секретами — утечка одного не подделывает остальные.
Сессии можно отозвать	Даже если токен технически ещё не истёк, запись о сессии в базе можно пометить «отозвана» — и доступ пропадёт мгновенно (выход, смена пароля, кнопка «Revoke»).
Человек видит свои устройства	В «Мой аккаунт» виден список активных сессий (система, браузер, время) — можно закрыть любую или все, кроме текущей.
Сброс пароля в два подтверждения	Сначала код из почты (одноразовый, 5 минут, 5 попыток), затем отдельный короткоживущий resetToken. После сброса выходят все устройства.
Смена пароля требует текущий пароль	Захват активной сессии сам по себе не даёт возможности тихо увести аккаунт себе. После смены остальные сессии закрываются, всё — в одной транзакции.
Строгая проверка входных данных	Любое поле, не предусмотренное конкретной формой, сервер отклоняет ещё до того, как оно попадёт в бизнес-логику.

Документ сгенерирован автоматически на основе реального прогона приложения (Playwright) поверх фактического кода бэкенда и фронтенда платформы. Разделы 1–6: 19 сентября 2026 г.; разделы 7–10: 29 сентября 2026 г.